

**A Project Report on**  
**AI Based Smart Resume Screening System**

Submitted in partial fulfilment for award of  
**Bachelor of Technology**

Degree

in

**CSE (Artificial Intelligence and Machine Learning)**

By

**Y. Bhuvanesh (Y22ACM489)**

**Sk. Shameem (Y22ACM472)**

**T. Chandana (Y22ACM480)**

**P. Punith Venkat Sai (Y22ACM492)**



Under the guidance of  
**Dr. Farheen Mohammed, M.S, Ph.D.,**  
Assistant Professor

Department of CSE (Artificial Intelligence and Machine Learning)

**Bapatla Engineering College**

(Autonomous)

(Affiliated to Acharya Nagarjuna University)

**BAPATLA – 522 102, Andhra Pradesh, INDIA**

**2025-2026**

**Department of  
CSE (Artificial Intelligence and Machine Learning)**



**CERTIFICATE**

This is to certify that the project report entitled **AI Based Smart Resume Screening System using Llama Model** being submitted by Y. Bhuvanesh (Y22ACM489), Sk. Shameem (Y22ACM472), T. Chandana (Y22ACM480), P. Punith Venkata Sai (Y22ACM492) in partial fulfillment for the award of the Degree of Bachelor of Technology in CSE (Artificial Intelligence and Machine Learning) to Acharya Nagarjuna University is a record of Bonafide work carried out by them under our guidance and supervision.

Date:

**Signature of the Guide**  
**Dr. Farheen Mohammed**  
**Assistant Professor**

**Signature of the HOD**  
**Dr. K. Manideep**  
**Assoc. Prof. & Head**

## **DECLARATION**

We declare that this project work is composed by ourselves, that the work contained herein is our own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

Yakkaladevi Bhuvanesh(Y22ACM489)

Shaik Shameem (Y22ACM472)

Telukutla Chandana (Y22ACM480)

Pentyala Punith Venkat Sai(Y22ACM492)

## Acknowledgement

We sincerely thank the following distinguished personalities who have given their advice and support for successful completion of the work.

We are deeply indebted to our most respected guide **Ms. Dr. Farheen Mohammed**, Assistant Professor, Department of CSE(AIML), for her valuable and inspiring guidance, comments, suggestions and encouragement.

We extend our sincere thanks to **Dr. K. Manideep**, Assoc. Prof. & Head, Department of CSE(AIML) for extending his cooperation and providing the required resources.

We would like to thank our beloved Principal **Dr. B.Srinivasa Rao** for providing the online resources and other facilities to carry out this work.

We would like to sincerely thank our project coordinator **Ms. G.V. Leela Kumari**, Assistant Prof., Dept. of CSE(AIML), for her helpful suggestions in presenting this document.

We extend our sincere thanks to all other teaching faculty and non-teaching staff of the department, who helped directly or indirectly for their cooperation and encouragement.

Yakkaladevi Bhuvanesh (Y22ACM489)

Shaik Shameem (Y22ACM472)

Telukutla Chandana (Y22ACM480)

Pentyala Punith Venkat Sai(Y22ACM492)

## ABSTRACT

The rapid increase in job applications has created significant challenges for human resource (HR) teams, making manual resume screening time-consuming, inconsistent, and prone to bias. Traditional screening methods primarily rely on keyword matching and static filtering techniques, which fail to capture the semantic meaning of candidate profiles and often lead to inaccurate shortlisting decisions. This project proposes an AI-Based Smart Resume Screening System that dynamically evaluates candidate resumes using advanced Natural Language Processing (NLP) and semantic understanding. The system processes inputs such as resumes, job descriptions, mandatory skills, preferred skills, and experience requirements. Based on this data, it computes multiple evaluation metrics including semantic similarity score, weighted skill matching score, and contextual experience score to effectively assess candidate suitability. Using LLaMA-based semantic embeddings, the system performs deep contextual matching between resumes and job descriptions. A power-law calibrated cosine similarity measure is applied to improve matching accuracy and reduce false positives. Additionally, the system incorporates a weighted skill extraction mechanism to prioritize mandatory skills and a contextual experience model to ensure relevance of candidate experience. To enhance transparency and trust, an Explainable AI (XAI) module is integrated, providing clear justifications for candidate rankings through skill matching insights, score breakdowns, and decision explanations. The system also includes an automated email notification module to inform candidates about their selection status. The application is implemented as a web-based platform with features such as candidate ranking, filtering, and performance tracking. Experimental results demonstrate improved accuracy, efficiency, and fairness compared to traditional screening methods.

**Keywords:** Artificial Intelligence, Resume Screening, LLaMA, Semantic Embeddings, Explainable AI, Recruitment Automation, Candidate Ranking.

## TABLE OF CONTENTS

|                                            |        |
|--------------------------------------------|--------|
| ABSTRACT.....                              | v      |
| LIST OF TABLES.....                        | viii   |
| LIST OF FIGURES .....                      | ix     |
| <br>CHAPTER 1: INTRODUCTION .....          | <br>1  |
| 1.1 Background of the Study.....           | 2      |
| 1.2 Problem Statement .....                | 2      |
| 1.3 Objectives .....                       | 2      |
| 1.4 Scope.....                             | 3      |
| 1.5 Organization of the Report.....        | 4      |
| <br>CHAPTER 2: LITERATURE REVIEW .....     | <br>5  |
| 2.1 Overview of Existing Work .....        | 5      |
| 2.2 Comparison of Previous Approaches..... | 7      |
| <br>CHAPTER 3: SYSTEM ANALYSIS.....        | <br>8  |
| 3.1 Existing System .....                  | 8      |
| 3.2 Proposed System .....                  | 9      |
| 3.3 Feasibility Study .....                | 10     |
| 3.4 Requirements .....                     | 10     |
| 3.4.1 Hardware Requirements.....           | 10     |
| 3.4.2 Software Requirements .....          | 11     |
| 3.4.3 Functional Requirements .....        | 11     |
| 3.4.4 Non-Functional Requirements .....    | 12     |
| <br>CHAPTER 4: SOFTWARE DESIGN .....       | <br>13 |
| 4.1 Architecture Design .....              | 13     |
| 4.2 UML Diagrams .....                     | 14     |
| 4.2.1 Class Diagram .....                  | 14     |
| 4.2.2 Interaction Diagrams.....            | 16     |
| 4.2.2.1 Sequence Diagram .....             | 16     |
| 4.2.2.2 Collaboration Diagram.....         | 17     |
| 4.2.3 Object Diagram .....                 | 18     |
| 4.2.4 Use Case Diagram.....                | 19     |
| 4.2.5 Activity Diagram.....                | 20     |
| 4.2.6 Control Flow Diagram .....           | 21     |
| 4.3 Database Design.....                   | 23     |
| 4.4 Algorithm / Design Methodology .....   | 25     |
| <br>CHAPTER 5: IMPLEMENTATION.....         | <br>26 |
| 5.1 Technologies Used.....                 | 26     |

|                                             |    |
|---------------------------------------------|----|
| 5.2 Modules Description .....               | 27 |
| 5.2.1 Resume Management Component .....     | 27 |
| 5.2.2 Semantic Intelligence Component ..... | 27 |
| 5.2.3 Candidate Evaluation Component .....  | 28 |
| 5.2.4 Explainable AI Component.....         | 29 |
| 5.2.5 Notification Component.....           | 30 |
| 5.2.6 Dashboard Component.....              | 30 |
| 5.2.7 Feedback & Learning Component .....   | 30 |
| 5.3 Coding & Implementation Details .....   | 31 |
| 5.3.1 Project Folder Structure .....        | 31 |
| 5.3.2 Backend Implementation .....          | 34 |
| 5.3.3 Frontend Implementation.....          | 38 |
| CHAPTER 6: TESTING.....                     | 57 |
| 6.1 Test Cases .....                        | 57 |
| 6.2 Testing Methods.....                    | 57 |
| 6.2.1 Unit Testing .....                    | 57 |
| 6.2.2 Integration Testing .....             | 58 |
| 6.2.3 System Testing.....                   | 59 |
| CHAPTER 7: RESULTS AND ANALYSIS .....       | 61 |
| 7.1 Results and Discussion .....            | 61 |
| 7.2 Output Screenshots .....                | 61 |
| 7.3 Performance Analysis .....              | 66 |
| 7.4 Comparison with Existing System.....    | 66 |
| CHAPTER 8: CONCLUSION AND FUTURE WORK ..... | 67 |
| 8.1 Summary of the Project.....             | 67 |
| 8.2 Achievements.....                       | 67 |
| 8.3 Limitations .....                       | 68 |
| 8.4 Future Enhancements.....                | 68 |
| REFERENCES .....                            | 69 |

## LIST OF TABLES

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| Table 2.1 Comparison of Existing AI-Based Resume Screening Systems ..... | 7  |
| Table 4.1 Job Configurations Schema .....                                | 23 |
| Table 4.2 User Session Schema.....                                       | 23 |
| Table 4.3 Screening History Schema.....                                  | 24 |
| Table 4.4 User Accounts Schema.....                                      | 24 |
| Table 6.1 Test Cases of AI-Based Resume Screening System.....            | 57 |
| Table 7.1 Comparison of Traditional and Proposed Systems.....            | 66 |



## LIST OF FIGURES

|                                                               |    |
|---------------------------------------------------------------|----|
| Fig 4.1 Architecture of AI-Based Resume Screening System..... | 14 |
| Fig 4.2.1 Class Diagram .....                                 | 16 |
| Fig 4.2.2.1 Sequence Diagram.....                             | 17 |
| Fig 4.2.2.2 Collaboration Diagram .....                       | 18 |
| Fig 4.2.3 Object Diagram .....                                | 19 |
| Fig 4.2.4 Use Case Diagram .....                              | 20 |
| Fig 4.2.5 Activity Diagram .....                              | 21 |
| Fig 4.2.6 Control Flow Diagram .....                          | 22 |
| Fig 5.3.1 Layered Execution Flow .....                        | 56 |
| Fig 5.3.2 File Interaction Flow .....                         | 57 |
| Fig 7.1.1 Home Interface of Resume Screening System .....     | 61 |
| Fig 7.1.2 Job Configuration Interface & Input .....           | 62 |
| Fig 7.1.3 AI Based Resume Input.....                          | 63 |
| Fig 7.1.4 AI Based Resume Processing.....                     | 63 |
| Fig 7.1.5 Shortlisted Candidates Dashboard .....              | 64 |
| Fig 7.1.6 Explainable AI Candidates Analysis .....            | 65 |
| Fig 7.1.6 Resume Screening History .....                      | 65 |



# CHAPTER 1

## INTRODUCTION

### 1.1 BACKGROUND OF THE STUDY

The rapid growth of online job portals and professional networking platforms has significantly increased the number of applications received for each job position. This makes manual resume screening time-consuming, inefficient, and prone to human errors. Traditional recruitment methods mainly rely on keyword-based filtering and manual review, which fail to capture the contextual meaning of candidate profiles and often result in inaccurate or biased shortlisting decisions.

Conventional resume screening systems are based on rule-based and keyword matching approaches. These methods do not effectively consider semantic relationships, skill relevance, or the contextual importance of experience, leading to poor evaluation accuracy. Additionally, such systems lack transparency, making it difficult for recruiters to understand the reasoning behind candidate selection or rejection.

With advancements in Artificial Intelligence and Natural Language Processing (NLP), it is now possible to develop intelligent systems that analyze resumes more effectively at a semantic level. The proposed **AI-Based Smart Resume Screening System** leverages LLaMA-based semantic embeddings to perform deep contextual matching between resumes and job descriptions. It also incorporates weighted skill extraction to prioritize mandatory and preferred skills, along with contextual experience scoring to evaluate candidate suitability more accurately.

Furthermore, the system integrates Explainable Artificial Intelligence (XAI) to provide clear and interpretable justifications for candidate ranking, improving transparency and trust in automated decisions. An automated email notification module is also included to efficiently communicate selection results to candidates. Overall, the system enhances accuracy, efficiency, and fairness, transforming traditional resume screening into an intelligent and user-centered recruitment solution.

## 1.2 PROBLEM STATEMENT

In the current recruitment process, organizations receive a large number of applications for each job opening, making manual resume screening a time-consuming and inefficient task. Recruiters often need to analyze hundreds or thousands of resumes, which increases workload and may lead to delays, human errors, and the possibility of overlooking qualified candidates.

Most existing resume screening systems rely on keyword-based filtering and rule-based approaches. These methods fail to capture the contextual meaning of resumes, including the relevance of skills, experience, and their alignment with job requirements. As a result, they often produce inaccurate shortlisting decisions, where suitable candidates may be rejected and less relevant candidates may be selected. Additionally, these systems lack transparency, making it difficult for recruiters to understand the reasoning behind candidate selection or rejection.

Although advancements in Artificial Intelligence and Natural Language Processing have introduced automated resume parsing and semantic matching techniques, many current systems still focus mainly on information extraction rather than complete candidate evaluation. They often lack integration of multiple evaluation factors such as semantic similarity, weighted skill prioritization, and contextual experience analysis.

Therefore, there is a need for an intelligent resume screening system that can efficiently process large volumes of resumes, perform deep semantic analysis, evaluate candidates using multiple criteria, and provide clear and explainable decision-making. Such a system will improve accuracy, reduce recruiter workload, and ensure a fair and efficient recruitment process.

## 1.3 OBJECTIVES

The primary objectives of the proposed **AI-Based Smart Resume Screening System** are:

1. To design and develop an automated resume screening system that efficiently processes large volumes of resumes and job descriptions for candidate shortlisting.
2. To implement advanced Natural Language Processing (NLP) techniques using LLaMA-based semantic embeddings to perform deep contextual matching between resumes and job requirements.

3. To develop a weighted skill extraction mechanism that distinguishes between mandatory and preferred skills for accurate candidate evaluation.
4. To design a multi-criteria scoring system that computes semantic similarity score, skill matching score, and contextual experience score for ranking candidates.
5. To incorporate adaptive scoring techniques such as power-law calibrated cosine similarity and alignment factors to improve matching accuracy and reduce false positives.
6. To integrate an Explainable Artificial Intelligence (XAI) module that provides clear justifications for candidate selection through score breakdowns, matched skills, and decision explanations.
7. To develop an automated email notification system that informs candidates about their selection or rejection status in a timely and professional manner.
8. To design an interactive web-based dashboard that enables recruiters to upload resumes, define job requirements, filter candidates, and view ranked results with explanations.

To evaluate the performance of the system by comparing it with traditional methods such as TF-IDF, NER-based models, and BERT-based approaches using metrics like accuracy, precision, recall, and F1-score

## 1.4 SCOPE

The proposed **AI-Based Smart Resume Screening System** provides an automated, web-based solution for evaluating and ranking resumes based on job requirements. It uses LLaMA-based semantic analysis, weighted skill matching, and contextual experience evaluation to improve the accuracy and efficiency of candidate shortlisting. The system also includes features such as resume parsing, explainable decision-making, candidate ranking, and automated email notifications.

The current implementation is designed for recruiter-side usage within a web environment, supporting standard resume formats and predefined criteria. Future enhancements include multi-user support, job portal integration, real-time notifications, and advanced model improvements. Thus, the system focuses on intelligent and automated resume screening within a scalable framework.

## **1.5 ORGANISATION OF THE REPORT**

This report is organized into multiple chapters for systematic presentation of the proposed AI-Based Smart Resume Screening System. Chapter 1 introduces the project including background, problem statement, objectives, and scope. Chapter 2 presents the literature review and comparison of existing approaches. Chapter 3 discusses system analysis including existing system, proposed system, feasibility study, and requirements. Chapter 4 explains software design including architecture, UML diagrams, database design, and methodology. Chapter 5 describes implementation details and technologies used. Chapter 6 presents testing methodologies and results. Chapter 7 illustrates output screens and performance results. Finally, Chapters 8 and 9 provide conclusion and future scope respectively.

The proposed system is designed to address the limitations of traditional resume screening methods by incorporating advanced AI techniques such as semantic analysis, similarity-based ranking, and Explainable AI. It focuses on improving recruitment efficiency by automating candidate evaluation while ensuring transparency and fairness in decision-making. The system leverages LLaMA-based embeddings to capture contextual meaning and provides detailed justifications for each recommendation, thereby assisting recruiters in making informed hiring decisions.

Furthermore, the project emphasizes scalability and adaptability to handle large volumes of resumes in real-time recruitment scenarios. It is structured to support continuous improvements through enhancements in model performance, integration of additional evaluation metrics, and inclusion of bias mitigation techniques. Overall, the system aims to provide a reliable, accurate, and user-friendly solution that enhances both recruiter productivity and candidate experience in modern hiring processes.

## **CHAPTER 2**

### **LITERATURE REVIEW**

#### **2.1 OVERVIEW OF EXISTING WORK**

Artificial Intelligence (AI) and Natural Language Processing (NLP) have played a significant role in transforming modern recruitment systems by enabling automated resume screening and intelligent candidate evaluation. Early approaches to resume screening relied on keyword-based filtering and rule-based parsing techniques, which extracted structured information from resumes using predefined patterns. Although these methods were computationally efficient, they lacked flexibility and failed to handle variations in resume formats and semantic expressions. To overcome these limitations, Named Entity Recognition (NER) techniques were introduced to identify key entities such as skills, job titles, and educational qualifications, thereby improving the accuracy of information extraction [3], [5]. Systems such as RINX further enhanced resume parsing by combining ontology-based reasoning with statistical models for better knowledge extraction [4].

With the advancement of deep learning, transformer-based models such as BERT have significantly improved the ability of systems to understand textual data at a contextual level. These models generate dense semantic embeddings that capture relationships between words and phrases, enabling more accurate matching between resumes and job descriptions [7], [8]. Additionally, recent approaches using large language models (LLMs) have demonstrated the recent research has shifted towards hybrid and end-to-end systems that combine multiple techniques such as semantic embeddings, skill extraction, and ranking algorithms. Systems like DeepResume and Smart-Hiring incorporate deep learning models along with Explainable Artificial Intelligence (XAI) to improve both the accuracy and transparency of candidate evaluation [12], [14]. AI-powered resume parsers integrated with ranking mechanisms have shown better performance compared to traditional TF-IDF and rule-based methods by leveraging contextual understanding and data-driven scoring techniques [6], [11]. These systems also address practical challenges such as handling large volumes of resumes and improving recruiter decision-making efficiency.

Despite these advancements, many existing systems primarily focus on resume parsing and information extraction, with limited emphasis on comprehensive candidate evaluation and ranking. Several approaches lack integration of semantic similarity, weighted skill prioritization, and contextual experience analysis within a unified framework. Furthermore, transparency in automated decision-making remains a major concern, as recruiters often require clear justifications for candidate selection or rejection. Therefore, there is a need for an intelligent resume screening system that combines semantic understanding, multi-criteria scoring, and explainable AI to provide accurate, efficient, and transparent recruitment solutions [9].

Recent developments in large language models such as LLaMA have further advanced the capabilities of resume screening systems by enabling deeper semantic understanding and contextual reasoning. Unlike earlier transformer models, LLaMA-based approaches can capture document-level relationships, allowing the system to interpret resumes and job descriptions more holistically. This leads to improved matching accuracy, especially in cases where candidates use diverse terminology or unconventional formats.

Another important area of research focuses on multi-criteria evaluation mechanisms that go beyond simple keyword or similarity matching. Modern systems incorporate factors such as skill relevance, experience alignment, educational background, and contextual importance of projects to generate a more comprehensive candidate score. Weighted scoring techniques and hybrid ranking models are increasingly being used to prioritize critical skills and domain-specific requirements. This enables recruiters to obtain a more balanced and realistic evaluation of candidates, improving the overall quality of hiring decisions.

Furthermore, the integration of Explainable Artificial Intelligence (XAI) has become essential in addressing trust and fairness concerns in automated recruitment systems. XAI techniques provide interpretable outputs such as feature importance, score breakdowns, and reasoning behind candidate rankings, allowing recruiters to understand and validate system decisions. This transparency not only enhances user confidence but also helps in identifying potential biases in the model. As a result, recent research emphasizes the development of end-to-end systems that combine semantic understanding, intelligent ranking, and explainability to create more reliable and ethical recruitment solutions.



## 2.2 COMPARISON OF PREVIOUS APPROACHES

Table 2.1: Comparison of Existing AI-Based Resume Screening Systems

| Study / Tool                          | Methodology                                                    | Limitation                                                       | Ref |
|---------------------------------------|----------------------------------------------------------------|------------------------------------------------------------------|-----|
| Keyword-Based Screening Systems       | Resume filtering using keyword matching and rule-based parsing | Cannot understand semantic meaning or contextual relevance       | [2] |
| Traditional ATS Platforms             | Rule-based resume parsing with predefined ranking filters      | May reject qualified candidates due to strict keyword dependency | [3] |
| Named Entity Recognition (NER) Models | Extraction of skills, education, and job roles from resumes    | Focuses mainly on data extraction, not candidate ranking         | [4] |
| RINX                                  | Ontology-based reasoning with statistical resume parsing       | Limited scalability and lacks advanced semantic evaluation       | [5] |
| BERT-Based Matching Systems           | Transformer embeddings for contextual resume-job matching      | High computational cost and limited explainability               | [6] |
| LLM Few-Shot Resume Parsers           | Prompt-based extraction using large language models            | Output consistency depends on prompt quality                     | [7] |
| DeepResume / Smart-Hiring             | Deep learning with ranking and explainable AI                  | Limited weighted skill prioritization and experience analysis    | [8] |

## **CHAPTER 3**

### **SYSTEM ANALYSIS**

#### **3.1 EXISTING SYSTEM**

Current resume screening methods primarily include manual review, keyword-based filtering, and basic Applicant Tracking Systems (ATS). These systems allow recruiters to sort resumes based on predefined keywords, qualifications, and experience levels.

However, most existing systems rely on static filtering techniques and do not consider the contextual meaning of resumes. They fail to capture semantic relationships between skills, experience, and job requirements. Additionally, these systems do not analyze candidate profiles holistically, leading to inaccurate shortlisting and potential bias in the recruitment process.

Furthermore, traditional systems lack adaptive intelligence and transparency. They do not provide clear reasoning for candidate selection or rejection and are unable to dynamically adjust evaluation criteria based on job requirements or candidate diversity.

#### **Limitations of Existing System**

The major limitations of current resume screening systems are:

- Dependence on keyword-based and rule-based filtering
- Lack of semantic understanding of resume content
- Inability to evaluate contextual experience and skill relevance
- No multi-criteria scoring or ranking mechanism
- Limited transparency in decision-making
- High chances of bias and inaccurate shortlisting
- Time-consuming manual intervention required
- No automated communication with candidates

## **3.2 PROPOSED SYSTEM**

The proposed system is an AI-Based Smart Resume Screening System designed to automate and improve the candidate evaluation process. Unlike traditional systems, it integrates semantic analysis, weighted skill matching, and contextual experience evaluation to provide accurate and efficient candidate shortlisting.

The system accepts resumes and job descriptions as input and processes them through multiple stages including resume parsing, preprocessing, semantic embedding generation using LLaMA-3, and similarity computation. A multi-criteria scoring mechanism evaluates candidates based on semantic similarity, skill matching, and experience relevance.

To enhance transparency, the system incorporates an Explainable Artificial Intelligence (XAI) module that provides clear reasoning for candidate ranking through score breakdown and matched skill analysis. Additionally, an automated email notification module is included to inform candidates about their selection status.

The system also features a web-based dashboard that allows recruiters to upload resumes, define job criteria, view ranked candidates, and analyze results efficiently. This approach transforms traditional resume screening into an intelligent, accurate, and transparent recruitment solution.

### **Advantages of Proposed System**

Semantic-based resume evaluation using LLaMA embeddings

- Accurate candidate ranking through multi-criteria scoring
- Weighted skill matching (mandatory and preferred skills)
- Contextual experience analysis for better evaluation
- Explainable AI for transparent decision-making
- Automated email notification system
- Reduced manual effort and faster screening process
- Improved accuracy and fairness in candidate selection
- Scalable and modular web-based implementation

### 3.3 FEASIBILITY STUDY

The feasibility study evaluates the practicality and viability of implementing the proposed AI-Based Smart Resume Screening System.

#### **Technical Feasibility**

The proposed system is technically feasible as it is developed using widely adopted modern technologies such as Python, React, LLaMA-based embedding models, FAISS, and machine learning libraries. These technologies provide sufficient flexibility and support for implementing resume parsing, semantic matching, candidate ranking, explainable AI, and interactive dashboards. The required development tools, frameworks, and computing resources are readily available and compatible with current systems.

#### **Economic Feasibility**

The project is economically feasible because it does not require expensive hardware or proprietary software licenses. Most of the tools and frameworks used are open-source and freely available. Deployment and maintenance costs are minimal since the system is web-based and can operate on standard computing devices or cloud platforms.

#### **Operational Feasibility**

The proposed system is operationally feasible because it is designed with a simple and user-friendly interface that can be easily operated by recruiters, HR teams, and organizations without requiring advanced technical expertise. The workflow of resume upload, job criteria definition, candidate ranking, and result analysis aligns with real-world recruitment practices, making it practical for industry adoption.

### 3.4 REQUIREMENTS

#### **3.4.1 Hardware Requirements**

The proposed **AI-Based Smart Resume Screening System** is a web-based application and does not require specialized hardware. The minimum hardware requirements for development and deployment are as follows:

- **Processor:** Intel Core i3 or higher
- **RAM:** Minimum 4 GB (8 GB recommended for development and model processing)

- **Storage:** Minimum 10 GB of available disk space
- **Internet Connection:** Stable broadband connection for data processing and communication
- **Display:** Standard monitor with minimum resolution of  $1366 \times 768$

For development and testing purposes, a system with higher processing power and memory is recommended to efficiently handle tasks such as resume parsing, semantic embedding generation, and model execution.

### 3.4.2 Software Requirements

The proposed **AI-Based Based Smart Resume Screening System** is developed using modern web and AI technologies. The required software components include:

- **Operating System:** Windows / Linux / macOS
- **Frontend Framework:** React.js
- **Programming Language:** Python (for backend), ReactJs/TypeScript (for frontend)
- **Backend Framework:** Flask / FastAPI
- **Runtime Environment:** Node.js
- **AI/ML Libraries:** PyTorch / Transformers (for LLaMA model), Scikit-learn
- **Text Processing Tools:** pdfplumber, PyMuPDF, OCR (Tesseract)
- **Database:** MongoDB / MySQL (optional based on implementation)
- **Code Editor:** Visual Studio Code (or equivalent IDE)
- **Web Browser:** Google Chrome / Mozilla Firefox
- **Version Control System:** Git (optional but recommended)

### 3.4.3 Functional Requirements

The functional requirements describe the core operations that the system must perform.

The system shall:

- Allow recruiters to upload resumes in PDF/DOC format
- Accept job description input along with mandatory and preferred skills
- Allow recruiters to define experience requirements and selection criteria
- Extract text and structured information from resumes

- Perform preprocessing including tokenization, stop-word removal, and normalization
- Generate semantic embeddings using the LLaMA-3 model
- Compute similarity between resumes and job descriptions using cosine similarity
- Apply power-law calibration to improve matching accuracy
- Identify and extract candidate skills using NER-based techniques
- Calculate weighted skill matching score based on mandatory and preferred skills
- Evaluate contextual experience relevance of candidates
- Generate a final composite score for each candidate
- Rank candidates based on their final scores
- Shortlist candidates based on a predefined threshold
- Provide Explainable AI outputs including matched skills and score breakdown
- Display ranked candidate list through an interactive dashboard
- Allow filtering and sorting of candidates based on various criteria
- Enable resume comparison between candidates
- Send automated email notifications to selected and rejected candidates
- Maintain logs of screening results and communication

### **3.4.4 Non-Functional Requirements**

The non-functional requirements describe the quality attributes of the system.

- **Performance:** The system shall process resumes and generate candidate rankings within acceptable response time, even for large volumes of data.
- **Usability:** The interface shall be user-friendly, intuitive, and responsive, enabling recruiters to easily upload resumes and analyze results.
- **Reliability:** The system shall provide consistent and accurate results without data loss or processing errors during screening.
- **Scalability:** The modular architecture shall support scaling to handle increasing numbers of resumes and integration with external recruitment platforms.
- **Maintainability:** The system shall be designed using modular and structured coding practices to allow easy updates, debugging, and feature enhancements.
- **Security:** The system shall implement input validation, secure data handling, and controlled access to protect sensitive candidate information.
- **Compatibility:** The application shall function correctly across major web browsers and different operating systems.

## **CHAPTER 4**

### **SOFTWARE DESIGN**

#### **4.1 ARCHITECTURE DESIGN**

The architecture of the proposed AI-Based Smart Resume Screening System is designed using a modular layered structure to ensure scalability, maintainability, and efficient interaction among system components. The system is divided into multiple interconnected modules including Resume & Job Description Input Layer, Resume Parsing & Data Extraction Layer, Text Preprocessing Layer, Semantic Embedding Layer, Similarity & Ranking Engine, Explainable AI Layer, and Recruiter Dashboard Interface.

The Resume & Job Description Input Layer collects resumes and job descriptions uploaded by recruiters in supported formats such as PDF and DOCX. The Resume Parsing & Data Extraction Layer extracts important candidate information such as personal details, skills, education, certifications, and work experience from uploaded resumes.

The Text Preprocessing Layer performs tokenization, stop-word removal, normalization, and lemmatization to prepare textual data for intelligent analysis. The Semantic Embedding Layer uses LLaMA-based models to generate contextual vector representations of resumes and job descriptions for accurate semantic comparison.

The Similarity & Ranking Engine computes similarity scores, weighted skill matching, and experience relevance to rank candidates according to job requirements. The Explainable AI Layer enhances transparency by providing score breakdowns, matched skills, candidate strengths, and justification for ranking decisions.

Finally, the Recruiter Dashboard Interface presents ranked candidates, analytics, filtering options, comparison tools, and automated email notification controls through an interactive and user-friendly web interface.

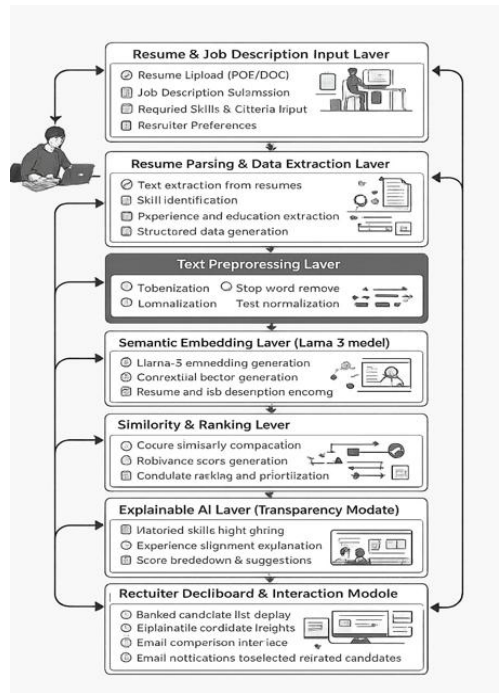


Fig 4.1: Architecture of the AI-Based Smart Resume Screening System

## 4.2 UML DIAGRAMS

Unified Modeling Language (UML) is a general-purpose modeling language. The main aim of UML is to define a standard way to visualize the way a system has been designed. It is quite similar to blueprints used in other fields of engineering. We use UML diagrams to show the behavior and structure of a system helps software engineers, businessmen, and system architects with modeling, design, and analysis.

### 4.2.1 Class Diagram

#### Objective:

Define the static structure of the AI-Based Smart Resume Screening System, including classes, their attributes, methods, and relationships between system components.

**JobDescription** – Represents the job requirements provided by the recruiter.

- **Attributes:** jobId, title, description, requiredSkills, criteria
- **Methods:** parseJD(), extractRequirements()

**TextProcessor** – Handles preprocessing of textual data.

- **Attributes:** rawText, tokens, normalizedText
- **Methods:** tokenize(), removeStopWords(), normalizeText()

**EmbeddingModel (LLaMA)** – Generates semantic embeddings for resumes and job descriptions.



- **Attributes:** modelName, modelVersion
- **Methods:** generateEmbeddings(), encodeText()

**SimilarityEngine** – Computes similarity between resume and job description embeddings.

- **Attributes:** similarityScore, threshold
- **Methods:** computeSimilarity(), compareVectors()

**RankingEngine** – Ranks candidates based on similarity scores.

- **Attributes:** rankedCandidates, averageScore
- **Methods:** rankCandidates(), generateRanking()

**ExplanationModule (XAI)** – Provides explanation for candidate evaluation.

- **Attributes:** matchedSkills, scoreBreakdown, confidence
- **Methods:** generateExplanation(), highlightMatches()

**RecruiterDashboard** – Interface for displaying results and managing candidates.

- **Attributes:** shortlistedCandidates, rejectedCandidates, screeningHistory
- **Methods:** displayResults(), filterCandidates(), exportResults()

**NotificationService** – Handles email communication to candidates.

- **Attributes:** selectedCandidates, rejectedCandidates, emailTemplates
- **Methods:** sendSelectionEmail(), sendRejectionEmail()

**Relationships:**

- TextProcessor processes data from Resume and JobDescription
- EmbeddingModel generates embeddings from processed text
- SimilarityEngine computes similarity using embeddings
- RankingEngine ranks candidates based on similarity scores
- ExplanationModule provides justification for ranking decisions
- RecruiterDashboard displays ranked and shortlisted candidates
- \* NotificationService sends emails based on final decisions

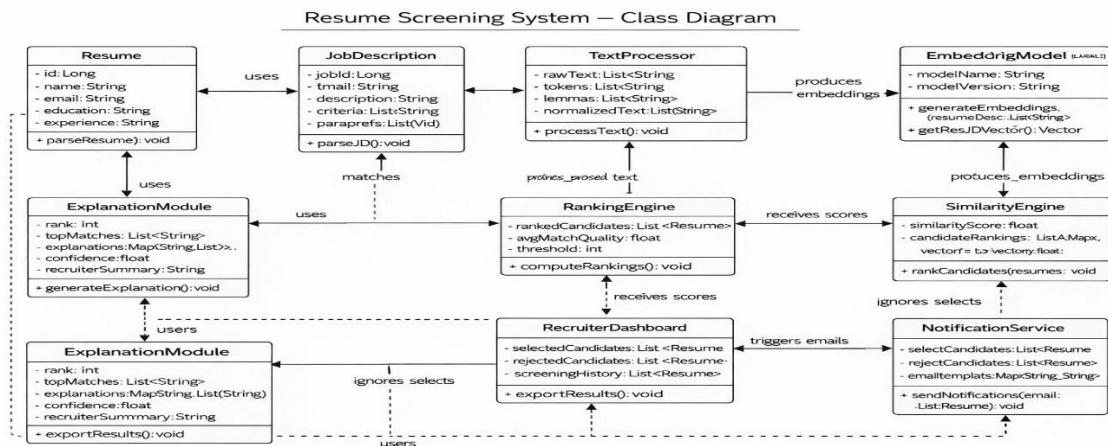


Fig 4.2.1. Class Diagram

### 4.2.2 Interaction Diagrams

#### 4.2.2.1 Sequence Diagram

### Objective:

Show system components interact in sequence to process resumes, evaluate candidates, and generate ranking with explanation..

- **Actors:** Recruiter, Dashboard Interface, Resume Parsing Module, Embedding Model, Scoring Engine, Explainable AI Module, Notification Service
- **Sequence:**
  1. **Recruiter** uploads resume and provide job description, required skills, and criteria through the Dashboard Interface.
  2. **Dashboard Interface** forwards the uploaded resumes and job description to the Resume Parsing Module.
  3. **Resume Parsing Module** extracts text, identifies skills, experience, and generates structured resume data.
  4. **Embedding Model** receives structured resume data and job description, and converts both into semantic embeddings.
  5. **Scoring Engine** computes similarity between resume and job description embeddings and evaluates candidates based on skill matching and experience relevance.
  6. **Scoring Engine** generates candidate scores and ranks candidates accordingly.
  7. **Explainable AI Module** generates explanations by highlighting matched skills, missing skills, and justification for candidate ranking.
  8. **Dashboard Interface** displays ranked candidates along with explanations and filtering options for the recruiter.

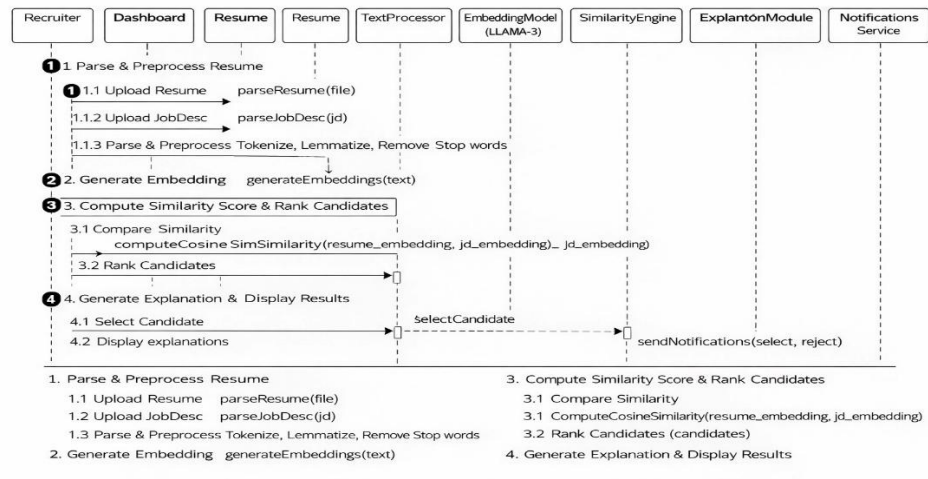


Fig.4.2.2.1 Sequence Diagram

#### 4.2.2.2 Collaboration Diagram

##### Objective:

Show how different system components interact and collaborate to perform automated resume screening, candidate evaluation, and decision-making.

##### Description:

**Actors:** Recruiter, Resume, TextProcessor, EmbeddingModel (LLaMA), SimilarityEngine, RankingEngine, ExplanationModule, RecruiterDashboard, NotificationService

##### Interactions:

- The recruiter uploads job requirements and candidate resumes into the system.
- The Resume component sends the uploaded data to the TextProcessor for parsing and preprocessing. The TextProcessor extracts relevant information and performs tokenization, normalization, and text cleaning.
- The processed text is passed to the EmbeddingModel (LLaMA), which generates semantic embeddings for both resumes and job descriptions.
- The SimilarityEngine computes similarity scores between resume embeddings and job description embeddings.
- The RankingEngine receives similarity scores and ranks candidates based on relevance and matching criteria.

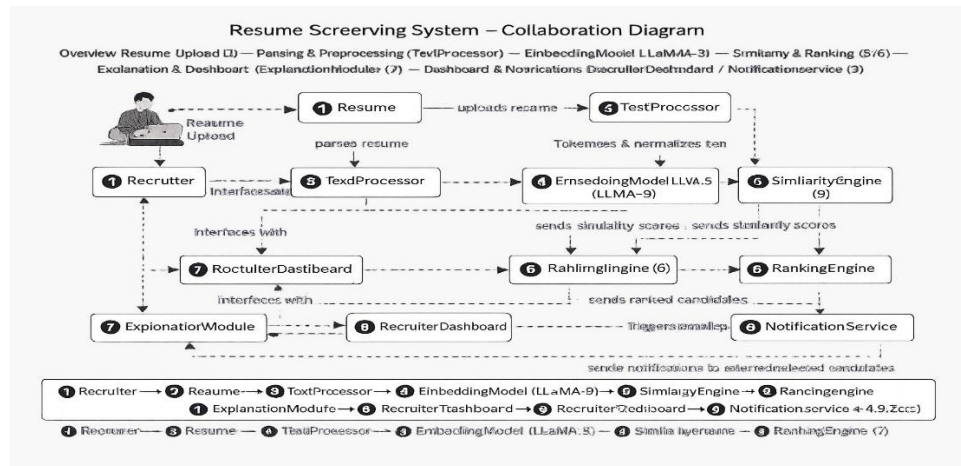


Fig. 4.2.2.2 Collaboration Diagram

## 4.2.3 Object Diagram

### Objective:

Represent a snapshot of objects in the AI-Based Smart Resume Screening System at a particular moment and illustrate how instances of system components interact to evaluate and rank candidates.

- **Objects:** Recruiter, Resume, JobDescription, Parser, EmbeddingModel, ScoringEngine, CandidateResult, ExplainableAI, Dashboard, NotificationService
- **Interactions:**
  1. **Recruiter object** uploads resume instances and provide job description details including required skills, experience, and criteria.
  2. **Resume objects** store candidate information such as skills, experience, education, and textual content extracted from documents.
  3. **JobDescription object** contains job-related information including role, required skills, and experience requirements used for candidate evaluation.
  4. **Parser object** processes resume instances by extracting text, identifying skills, and converting them into structured data format.
  5. **EmbeddingModel object** converts both resume data and job description into semantic embeddings for contextual comparison.
  6. **ScoringEngine object** evaluates candidates by computing similarity scores, skill matching, and experience relevance to generate ranking scores.
  7. **CandidateResult object** represents the evaluated output including candidate score, rank, and selection status.

8. **ExplainableAI object** enhances the results by providing explanations such as matched skills, missing skills, and justification for ranking.
9. **Dashboard object** displays ranked candidates along with explanations and allows filtering and comparison of resumes.
10. **NotificationService object** sends email notifications to candidates indicating selection or rejection along with explanation details.

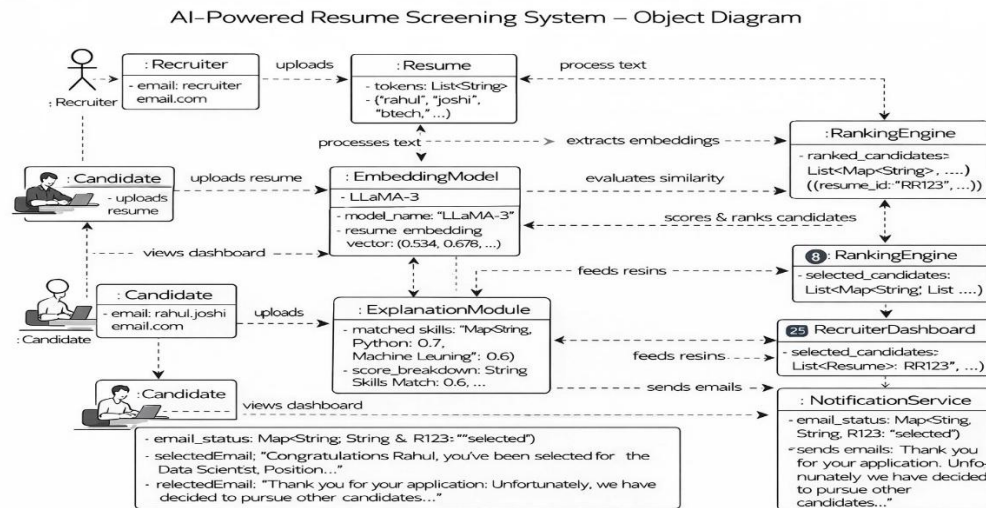


Fig. 4.2.3 Object Diagram

## 4.2.4 Use Case Diagram

Illustrate the functional interactions between the recruiter and the AI-Based Smart Resume Screening System by representing the key system functionalities

- **Actor:** Recruiter
- **Use Cases:**
  1. **Upload Resumes** – The recruiter uploads candidate resumes in supported formats (PDF/DOC) for processing.
  2. **Provide Job Description** – The recruiter enters job details including role, required skills, experience, and evaluation criteria.
  3. **Manage Screening Criteria** – The recruiter defines or updates required and preferred skills, weightage, and selection parameters.
  4. **Process and Parse Resumes** – The system extracts text, identifies skills, and converts resumes into structured data.
  5. **Generate Semantic Embeddings** – The system converts both resumes and job descriptions into embeddings for contextual understanding.

6. **Evaluate and Rank Candidates** – The system computes similarity scores, evaluates skill relevance and experience, and ranks candidates accordingly.
7. **View Candidate Rankings** – The recruiter views the ranked list of candidates based on evaluation scores.
8. **View Explainable Results** – The system provides explanations including matched skills, missing skills, and justification for rankings.
9. **Filter and Compare Candidates** – The recruiter filters candidates based on criteria and compares multiple resumes.
10. **Send Notification Emails** – The system sends emails to candidates indicating selection or rejection along with explanation details.

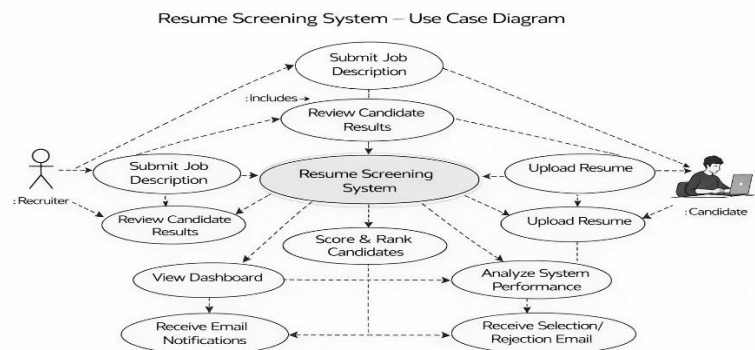


Fig. 4.2.4 Use Case Diagram

### 4.2.5 Activity Diagram

Represent the workflow of the AI-Based Smart Resume Screening System by illustrating the sequence of activities involved in processing resumes, evaluating candidates, and generating ranked results with explanations.

- **Activities:** Resume Upload, Job Description Input, Resume Parsing, Embedding Generation, Candidate Evaluation, Explanation Generation, Result Display, Notification Sending
- **Flow:**
  1. **Recruiter starts the application** and uploads resumes along with job description details including required skills, experience, and evaluation criteria.
  2. The **Dashboard Interface** receives the input and forwards resumes and job description to the processing modules.
  3. The **Resume Parsing Module** extracts textual content from resumes, identifies skills and experience, and converts them into structured data.

4. The **Embedding Model** processes both structured resume data and job description to generate semantic embeddings for contextual understanding.
5. The **Scoring Engine** computes similarity between embeddings, evaluates skill relevance and experience, and generates candidate scores and rankings.
6. The **Explainable AI Module** generates explanations by highlighting matched skills, missing skills, and justification for candidate ranking.
7. The **Dashboard Interface** displays ranked candidates along with explanations, allowing the recruiter to filter and analyze results.
8. The **Notification Service** sends email notifications to candidates indicating selection or rejection along with explanation details.
9. The **system updates evaluation data**, enabling improved accuracy and decision-making for future resume screening processes

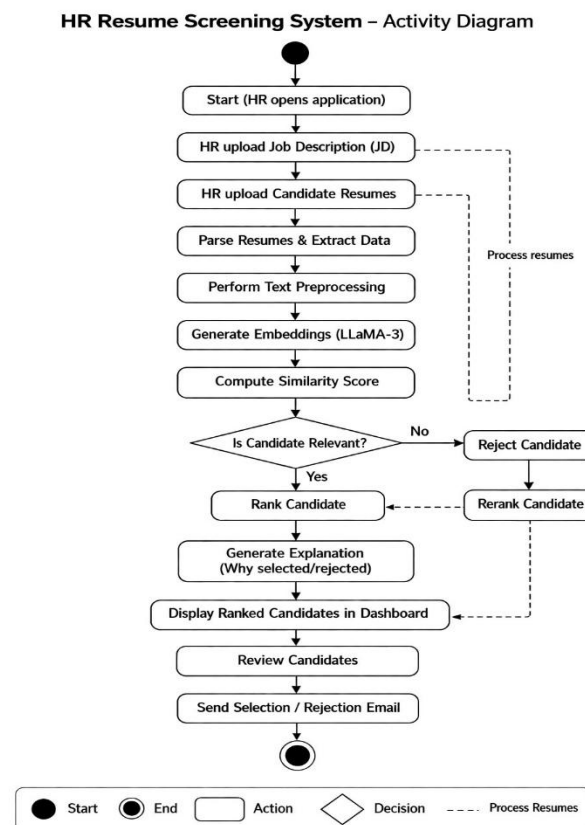


Fig. 4.2.5 Activity Diagram

## 4.2.6 Control Flow Diagram

Represent the flow of control within the AI-Based Smart Resume Screening System from recruiter input to candidate evaluation and result visualization.

### Description:

1. **Recruiter enters input** by uploading resumes and providing job description details such as required skills, experience, and evaluation criteria through the dashboard interface.
2. The **Dashboard Interface** receives the input and forwards resumes and job description to the Resume Parsing Module.
3. The **Resume Parsing Module** processes resumes by extracting text, identifying skills and experience, and converting them into structured data.
4. The **Embedding Model** receives structured resume data and job description, and converts both into semantic embeddings for contextual comparison.
5. The **Scoring Engine** evaluates candidates by computing similarity scores, analyzing skill relevance and experience, and generating candidate rankings.
6. The **Explainable AI Module** enhances the results by generating explanations, including matched skills, missing skills, and justification for ranking decisions.
7. The **Dashboard Interface** displays ranked candidates along with explanations, enabling the recruiter to analyze and filter results.
8. The **Notification Service** sends email notifications to candidates indicating selection or rejection along with explanation details.

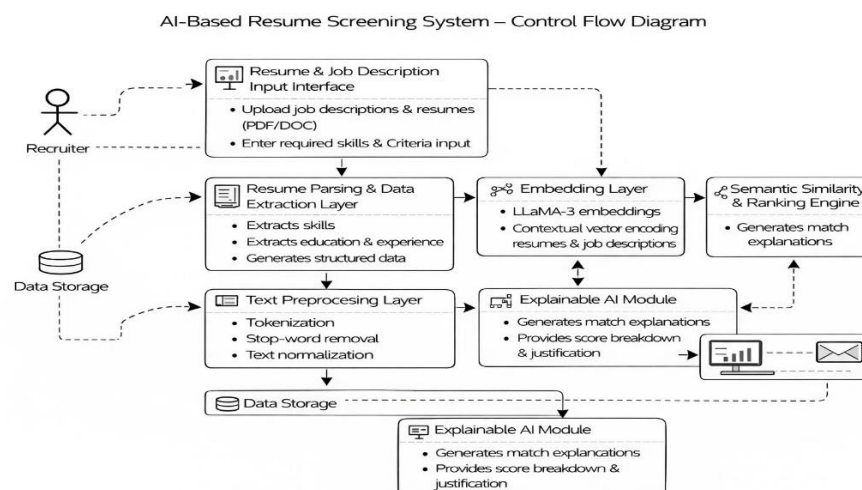


Fig. 4.2.6 Control Flow Diagram



### 4.3 DATABASE DESIGN

The proposed AI-Based Smart Resume Screening System utilizes a cloud-based relational database to securely manage and persist application data. Unlike systems limited to browser-based storage, this approach enables cross-device synchronization, secure candidate information handling, long-term record maintenance, and efficient recruitment workflow management.

The primary data entities stored within the system include Job Configurations, User Sessions, Screening History, Shortlisted Candidates, and User Accounts.

**Table 4.1 Job Configurations Schema**

| Field Name   | Data Type | Description                             |
|--------------|-----------|-----------------------------------------|
| id           | String    | Unique identifier for job configuration |
| user_id      | String    | Recruiter/User reference ID             |
| job_title    | String    | Position title                          |
| dueDate      | Date      | Deadline of task                        |
| priority     | String    | Priority level                          |
| timeEstimate | Number    | Estimated duration                      |
| completed    | Boolean   | Completion status                       |

**Table 4.2 User Session Schema**

| Field Name | Data Type | Description              |
|------------|-----------|--------------------------|
| user_id    | String    | Unique recruiter ID      |
| job_data   | JSON      | Active job settings      |
| results    | JSON      | Resume screening results |
| updated_at | Date      | Last updated timestamp   |

**Table 4.3 Screening History Schema**

| Field Name        | Data Type | Description                   |
|-------------------|-----------|-------------------------------|
| id                | String    | Unique screening batch ID     |
| user_id           | String    | Recruiter reference ID        |
| job_config_id     | String    | Job configuration reference   |
| job_title         | String    | Job title snapshot            |
| threshold         | Number    | Minimum eligibility score     |
| shortlisted_count | Number    | Number of selected candidates |
| full_results      | JSON      | Complete screening output     |
| created_at        | Date      | Screening timestamp           |

**Table 4.4 User Accounts Schema**

| Field Name         | Data Type | Description            |
|--------------------|-----------|------------------------|
| id                 | String    | Unique user ID         |
| email              | String    | Recruiter email        |
| encrypted_password | String    | Secured password       |
| created_at         | Date      | Registration timestamp |

## 4.4 ALGORITHM / DESIGN METHODOLOGY

The proposed system follows an intelligent resume screening algorithm that dynamically evaluates candidates based on semantic similarity, skill relevance, and experience suitability.

### Screening Methodology

1. Recruiter uploads resume and enters job description, required skills, and hiring criteria.
2. The Resume Input Module stores resumes and job requirement details.
3. The Resume Parsing Module extracts candidate information such as skills, education, certifications, and experience.
4. The Text Preprocessing Module performs cleaning, tokenization, normalization, and stop-word removal.
5. The Semantic Embedding Module generates vector embeddings for resumes and job descriptions using LLaMA-3.
6. The Similarity Engine computes semantic similarity between candidate profiles and job requirements.
7. Weighted skill matching and experience relevance scores are calculated based on mandatory and preferred criteria.
8. Final candidate scores are generated, ranked, and displayed on the recruiter dashboard.
9. The Explainable AI Module provides score breakdown, matched skills, and ranking justification.
10. Automated email notifications are sent to selected or rejected candidates.

This algorithm ensures accurate candidate shortlisting while improving transparency, efficiency, and fairness in recruitment.

## **CHAPTER 5**

### **IMPLEMENTATION**

#### **5.1 TECHNOLOGIES USED**

The proposed AI-Based Smart Resume Screening System is developed using modern web technologies, scalable backend architectures, and AI-integrated machine learning frameworks to ensure fast processing, reliable analytics, and intelligent candidate evaluation. The technologies used in the implementation of the system are described below:

##### **React.js**

React.js is used as the frontend JavaScript library for building the user interface of the application. It enables component-based development, efficient rendering, dynamic recruiter interactions, and responsive dashboards for resume screening.

##### **Python**

Python serves as the primary programming language for backend development. Its extensive ecosystem of machine learning, NLP, and data processing libraries makes it suitable for resume parsing, candidate scoring, and semantic analysis.

##### **LLaMA-3 / Large Language Model Integration**

LLaMA-3 is integrated into the system for semantic embedding generation and contextual understanding of resumes and job descriptions. It improves candidate-job matching accuracy through deep language understanding.

##### **FAISS**

FAISS (Facebook AI Similarity Search) is used for efficient similarity search and vector comparison. It helps in fast matching of resume embeddings with job description embeddings for ranking candidates.

##### **Sentence-Transformers**

Sentence-Transformers is used to generate dense semantic vector embeddings from resumes and job descriptions. It supports contextual similarity analysis for intelligent screening.

##### **PyTorch**

PyTorch is used as the deep learning framework for executing transformer-based embedding models and handling AI computations efficiently.

### **PDFPlumber / Document Parsing Libraries**

PDFPlumber and related parsing libraries are used for extracting clean text from PDF resumes and other document formats. They ensure accurate retrieval of candidate information such as skills, education, and experience.

### **NumPy**

NumPy is used for numerical computations, matrix operations, and similarity score calculations during candidate evaluation.

### **HTML / CSS**

HTML and CSS are used for structuring and styling the application interface to provide a clean, responsive, and user-friendly recruiter experience.

### **Visual Studio Code**

Visual Studio Code is used as the integrated development environment (IDE) for coding, debugging, and project management during system development.

## **5.2 MODULES DESCRIPTION**

### **5.2.1 Resume Management Component (Data Input Layer)**

Among the different elements of the system, the Resume Management Component acts as the system's data repository. The recruiter provides candidate resumes and job description details, and this module collects and maintains the data. It ensures that all inputs are stored in a standardized format for use by other modules of the system.

Resumes are stored using a structured format such as: (Resume ID, Candidate Name, Skills, Experience, Education, and Status), while job descriptions include (Role, Required Skills, and Experience Criteria). Functions include uploading resumes, managing job descriptions, updating candidate information, and organizing input data. This module enables efficient handling and retrieval of data for further processing and analysis.

### **5.2.2 Semantic Intelligence Component**

The Semantic Intelligence Component utilizes advanced Natural Language Processing techniques to understand and interpret resume content and job descriptions. It converts textual data into meaningful representations using embedding models.

The system generates semantic embeddings for both resumes and job descriptions to capture contextual meaning rather than relying on simple keyword matching. These embeddings allow the system to evaluate the relevance of candidate profiles more accurately.

### 5.2.3 Candidate Evaluation and Ranking Component (Core Intelligence Engine)

The core intelligence of the proposed system is provided by the Candidate Evaluation and Ranking Component. The system is designed to evaluate and rank candidate resumes by analyzing semantic similarity, skill relevance, and experience alignment with the given job description. The process begins with retrieving structured resume data from the Resume Parsing Component along with job description inputs.

Subsequently, both resume and job description data are converted into semantic embeddings using the Embedding Model. These embeddings capture contextual meaning and enable accurate comparison beyond simple keyword matching.

#### **Assessment of Candidate Relevance:**

A relevance weight  $W_i$  is assigned to each candidate based on the importance of matched skills, required experience, and job-specific criteria. Candidates with higher alignment to required skills are given greater importance during ranking.

#### **Semantic Similarity Evaluation:**

The system computes similarity between resume embeddings and job description embeddings using cosine similarity. This ensures that candidates are evaluated based on contextual relevance rather than exact keyword matches.

#### **Skill-Based Weight Adjustment:**

The system prioritizes mandatory and preferred skills by assigning higher weights to critical skills. Candidates possessing essential skills are ranked higher compared to those with partial matches.

#### **Experience-Based Evaluation:**

Candidate experience is analyzed in relation to required experience. Higher relevance is assigned to candidates whose experience closely matches or exceeds the job requirements.

#### **Contextual Relevance Analysis:**

The system evaluates how well the candidate's profile aligns with the job role by considering the overall semantic context of skills, projects, and experience.

To determine the final candidate ranking, the system computes a unified evaluation score:

$$S_i = W_i \times (\lambda_1 SS + \lambda_2 SM + \lambda_3 ER)$$

Where:

- $S_i$  represents the final score of candidates  $i$
- SS represents semantic similarity score
- SM represents skill matching score
- ER represents experience relevance score
- $\lambda_1, \lambda_2, \lambda_3$  are weighting coefficients controlling the influence of each factor

Candidates with higher evaluation scores are prioritized during ranking. The system ensures that the most relevant candidates are shortlisted based on a combination of semantic understanding, skill alignment, and experience relevance.

The Candidate Evaluation and Ranking Component serves as the central intelligence of the system, responsible for accurately evaluating and ranking candidates based on their relevance to the given job description. This component operates by comparing semantic embeddings of resumes with job description embeddings, enabling contextual understanding of candidate profiles beyond simple keyword matching.

The system computes similarity scores using techniques such as cosine similarity, ensuring that the evaluation captures the meaning and intent of the content. In addition to semantic similarity, the system applies weighted scoring based on critical factors such as skill relevance, experience alignment, and job-specific requirements. Mandatory skills are assigned higher importance, while additional skills contribute to overall candidate strength. Furthermore, experience is evaluated to determine how closely it aligns with the required experience level. The combined influence of these factors results in a final evaluation score for each candidate. Based on these scores, candidates are ranked in descending order, ensuring that the most suitable candidates are shortlisted. This component ensures a fair, consistent, and data-driven evaluation process, significantly improving recruitment efficiency.

#### **5.2.4 Explainable AI Component**

The Explainable AI Component plays a crucial role in enhancing the transparency and interpretability of the system. It provides detailed reasoning behind candidate evaluation results, enabling recruiters to understand how decisions are made.

This component analyzes the evaluation outputs and generates explanations by identifying matched skills, missing skills, and the overall alignment of the candidate profile with job requirements. It highlights the key factors that contributed to the candidate's score and ranking position.

By presenting clear and structured explanations, this module reduces ambiguity and builds trust in the system. It allows recruiters to validate decisions and ensures that the selection process is transparent, interpretable, and justifiable.

### **5.2.5 Notification and Communication Component**

The Notification and Communication Component is responsible for delivering evaluation outcomes to candidates in an efficient and automated manner. Once the candidate ranking process is completed, this module generates and sends email notifications indicating whether a candidate is selected or rejected.

Communication includes not only the decision but also relevant explanation details, providing candidates with clarity about the outcome. This improves the overall candidate experience and ensures transparency in the recruitment process.

By automating communication, this component reduces manual effort, saves time for recruiters, and ensures timely updates to all applicants.

### **5.2.6 Dashboard and Interaction Component**

The Dashboard and Interaction Component acts as the primary interface through which recruiters interact with the system. It provides a user-friendly environment for managing the entire resume screening process.

Recruiters can upload resumes, input job descriptions, and view ranked candidate lists along with detailed explanations. The dashboard includes features such as filtering, sorting, and comparison of candidates, allowing efficient analysis of large applicant pools.

Additionally, the dashboard presents results in a structured and visually understandable format, helping recruiters make quick and informed decisions. This component ensures smooth interaction between the user and the system while enhancing usability and productivity.

### **5.2.7 Feedback and Learning Component**

The Feedback and Learning Component enables the system to continuously improve its performance over time. It records evaluation results, recruiter interactions, and candidate outcomes to analyze patterns and refine the screening process.

By utilizing this feedback, the system can adjust scoring parameters, improve ranking accuracy, and enhance decision-making capabilities. This adaptive mechanism ensures that the system evolves with usage and becomes more effective in identifying suitable candidates.



Over time, the system becomes more intelligent and reliable, contributing to a more efficient and optimized recruitment process.

## **5.3 CODING**

### **5.3.1 Project Folder Structure**

The project is organized as a full-stack AI resume screening platform with a Python backend and a React frontend. The backend handles authentication, resume parsing, NLP extraction, semantic matching, scoring, explainability, exports, history persistence, and notifications. The frontend provides authentication, job configuration, resume upload and processing workflow, candidate shortlisting, explainability views, CSV export, and history management. This modular structure helps in better development, maintenance, and future enhancement of the application.

#### **A. Backend**

The backend module contains the server-side logic of the system. It handles all AI-related services, resume processing, scoring computation, explainability, communication, and API communication.

#### **Files included in Backend:**

- api/server.py
- supabase\_client.py
- core/config.py
- core/text\_extractor.py
- core/nlp\_engine.py
- core/skill\_extractor.py
- core/experience\_extractor.py
- core/qualification\_extractor.py
- core/contact\_extractor.py
- core/embedding\_engine.py
- core/scoring.py

core/hybrid\_extractor.py

core/xai\_engine\_v3.py

core/communication\_engine.py

core/notification\_engine.py

## **B. Frontend**

The frontend module contains the client-side part of the application. It is responsible for displaying the resume screening interface, managing navigation between pages, handling recruiter interactions, and presenting AI-generated shortlisting results and explanations to the user.

### **Main files included in Frontend:**

frontend/src/main.jsx

frontend/src/App.jsx

frontend/src/api/client.js

frontend/src/context/AuthContext.jsx

### **Frontend Components:**

frontend/src/components/Navbar.jsx

frontend/src/components/ProtectedRoute.jsx

### **Frontend Pages:**

frontend/src/pages/Home.jsx

frontend/src/pages/Login.jsx

frontend/src/pages/JobConfig.jsx

frontend/src/pages/Processing.jsx

frontend/src/pages/Candidates.jsx

frontend/src/pages/History.jsx

## C. Utilities and Data

This layer stores persistent settings, skills taxonomy, screening history, labels, and helper logic.

### Key files:

utils/export\_utils.py

utils/history\_store.py

utils/remember\_email.py

data/config.json

data/skills.json

data/history.json

data/ground\_truth.json

supabase\_setup.sql

## D. Styling

Styling is modular, with one global stylesheet plus page/component-specific CSS files.

### Style files:

frontend/src/index.css

frontend/src/components/Navbar.css

frontend/src/pages/Home.css

frontend/src/pages/Login.css

frontend/src/pages/JobConfig.css

frontend/src/pages/Processing.css

frontend/src/pages/Candidates.css

frontend/src/pages/History.css

### 5.3.2 Backend

**File: api/server.py**

**Purpose:**

Main Flask API server and orchestration layer. This is the central backend controller that wires all core modules together into REST APIs and returns structured JSON responses.

**Main Functions:**

Handles authentication routes using Supabase.

Saves and validates job configuration.

Processes uploaded resumes in parallel using thread pool workers.

Performs duplicate file detection via MD5 hash.

Stores and retrieves session data.

Exposes results, XAI, CSV export, email draft, notification, and history endpoints.

**Error Handling:**

Uses route-level validation and try/except blocks, with graceful fallbacks where possible.

**File: supabase\_client.py**

**Purpose:**

Initializes the Supabase client from environment variables. Serves as the shared database and authentication connectivity point for the entire backend.

**Main Functions:**

Loads URL and anon key from environment.

Creates a singleton client instance.

Returns a None-safe state when credentials are unavailable.

**File: core/config.py**

**Purpose:**

Central scoring and LLM configuration loader. Acts as the global runtime configuration source for scoring and AI behavior.

**Main Functions:**

- Loads JSON config from the data folder.
- Resolves the active penalty profile.
- Exposes score weights and thresholds as constants.
- Resolves Groq API key from environment or config.

**File: core/nlp\_engine.py**

**Purpose:** NLP utility engine for entity parsing and candidate name inference. Used by extraction layers for quality improvement.

**Main Functions:**

- Loads spaCy model with fallback model choices.
- Extracts potential skill entities.
- Validates skill entities using fuzzy matching.
- Derives candidate names from structured and unstructured resume text.

**File: core/skill\_extractor.py****Purpose:**

Multi-layer skill extraction and normalization module. Produces normalized candidate skills for scoring and explainability.

**Main Functions:**

- Loads canonical skills and synonyms from the skills dataset.
- Performs direct regex matching.
- Applies synonym expansion.
- Applies fuzzy matching and n-gram checks.
- Optionally enriches results via NLP engine entity validation.

**File: core/experience\_extractor.py****Purpose:**

Extracts professional experience duration and project or role context. Supplies experience metrics to the scoring engine.

**Main Functions:**

Detects experience sections and date intervals.

Passes multiple data formats.

Distinguishes professional roles from project-only contexts.

Estimates total years of valid professional experience.

**File: core/qualification\_extractor.py****Purpose:**

Extracts and validates educational qualification details. Acts as the qualification gate in candidate eligibility evaluation.

**Main Functions:**

Detects degree level patterns.

Extracts education block and year references.

Matches candidate qualification against required qualification and year conditions.

**File: core/contact\_extractor.py****Purpose:**

Extracts contact and profile information from resumes. Used for candidate identity and contact enrichment during shortlist actions.

**Main Functions:**

Email extraction including obfuscated patterns.

Phone extraction and normalization.

Name extraction from top resume segments.

LinkedIn, GitHub, and portfolio URL extraction.

**File: core/embedding\_engine.py****Purpose:**

Compute semantic embeddings for job descriptions and resumes. Serves as the foundation of semantic relevance scoring.

**Main Functions:**

- Uses Groq embedding API when available.
- Falls back to a local sentence-transformers model.
- Normalizes vectors for stable cosine similarity.
- Expose resume and query embedding methods.

**File: core/scoring.py****Purpose:**

Computes the final candidate score and detailed breakdown. Acts as the core ranking logic used to shortlist or reject candidates.

**Main Functions:**

- Semantic similarity scoring with power-law calibration.
- Scoring weighted skill with must-have emphasis.
- Experience fit scoring and contextualization.
- Qualification contribution scoring.
- Alignment factor and additional penalty composition.
- Confidence score calculation from extraction quality signals.

**File: core/hybrid\_extractor.py****Purpose:**

LLM fallback extractor for low-confidence resume cases. Acts as a reliability booster for difficult resume formats.

**Main Functions:**

- Calls an LLM to extract structured candidate fields.

Merges heuristic and LLM outputs safely.

Improves robustness when name or skills are missing from rule-based extraction.

**File:** `core/communication_engine.py`

**Purpose:**

Generates recruitment email drafts via LLM. Automates candidate communication for shortlisted and rejected candidates.

**Main Functions:**

Drafts next-steps emails for shortlisted candidates.

Drafts rejection emails for non-selected candidates.

Adapts language tone and brevity by email type.

**File:** `core/notification_engine.py`

**Purpose:**

Sends candidate emails through SMTP. Acts as the operational outbound communication layer.

**Main Functions:**

Loads SMTP configuration.

Generates or falls back to template content.

Sends emails and returns a sent, failed, and skipped summary.

### 5.3.3 Frontend

**File:** `frontend/src/main.jsx`

**Purpose:**

Frontend bootstrap entry point. Acts as the application startup file.

**Main Functions:**

Creates the React root.

Wraps the app with the authentication context provider.

Enables strict mode checks in development.



**File: frontend/src/App.jsx**

**Purpose:**

Route registry and app shell composition. Registers all public and protected routes and renders the global navbar.

**Main Functions:**

Registers public and protected routes.

Renders the global navigation bar.

Guards core pages through ProtectedRoute.

**Routes:**

/ → Home | /login → Login | /job-config → JobConfig | /processing → Processing | /candidates → Candidates | /history → History

**File: frontend/src/api/client.js**

**Purpose:**

Central API client abstraction. Acts as the single communication layer between the UI and the Flask backend.

**Main Functions:**

Configures axios base URL and JSON headers.

Injects bearer token via interceptor.

Exposes typed helper calls for auth, processing, results, XAI, email, notifications, export, and history APIs.

Handles CSV blob download logic.

**File: frontend/src/context/AuthContext.jsx**

**Purpose:**

Global authentication state management. Provides shared auth state across all pages and components.

**Main Functions:**

Maintains user, session, and loading state.

Persists and restores auth data from local storage.

Exposes login and logout handlers.

**File:** `frontend/src/components/Navbar.jsx`

**Purpose:**

Global responsive navigation component. Provides unified app navigation and user controls.

**Main Functions:**

Displays route links based on authentication state.

Highlights the active route.

Supports mobile menu behavior.

Triggers logout and redirection.

**File:** `frontend/src/pages/Home.jsx`

**Purpose:**

Landing page and feature overview. Acts as the entry UX and onboarding page of the application.

**Main Functions:**

Introduces system capabilities to new users.

Displays a workflow explanation.

Routes the user to the start flow.

**File: frontend/src/pages/Login.jsx**

**Purpose:**

Sign-in and sign-up interface. Acts as the user authentication interface.

**Main Functions:**

Handles login and signup forms.

Performs basic input validation.

Stores remembered email preference.

Handles backend error and success messaging.

**File: frontend/src/pages/JobConfig.jsx**

**Purpose:**

Job requirement configuration form. Defines the input criteria for the resume screening process.

**Main Functions:**

Captures title, description, qualifications, passing years, experience, must-have and good-to-have skills.

Validates JD word count and required fields.

Supports predefined and custom skills.

Saves configuration and navigates to the processing flow.

**File: frontend/src/pages/Processing.jsx**

**Purpose:**

Resume upload and screening trigger page. Acts as the execution stage for screening jobs.

**Main Functions:**

Supports drag-drops and file browse upload.

Accepts PDF, DOCX, TXT, and image files.

Extracts ZIP archives client-side before upload.

Calls the backend processing API with multipart form data.

**File:** `frontend/src/pages/Candidates.jsx`

**Purpose:**

Candidate review, explainability, comparison, and action page. Serves as the decision-support interface for recruiters.

**Main Functions:**

Loads screening results from the session API.

Provides fallback local analysis when API explainability fails.

Supports candidate comparison, email draft generation, notifications, and CSV export.

**File:** `frontend/src/pages/History.jsx`

**Purpose:**

Historical screening record explorer and manager. Acts as the audit trail and workflow continuity page.

**Main Functions:**

Loads all past screening sessions.

Expands records to inspect details and shortlisted candidates.

Restores prior results into the active session for review.

Supports re-screening setup reuse.

Supports single delete and clear-all operations.

### 5.3.4 Frontend Components and Styling Summary

**Reusable core components:**

`frontend/src/components/Navbar.jsx`

`frontend/src/components/ProtectedRoute.jsx`

**Global and page styling:**

frontend/src/index.css

frontend/src/components/Navbar.css

frontend/src/pages/Home.css

frontend/src/pages/Login.css

frontend/src/pages/Processing.css

frontend/src/pages/Candidates.css

frontend/src/pages/History.css

**Style Notes:**

Uses a dark, glassmorphism-inspired visual language. Maintains responsive layouts across desktop and mobile breakpoints. Applies separate CSS modules per page for maintainability.

**5.3.5 Infrastructure and Configuration Files****Backend and runtime:**

requirements.txt

data/config.json

data/skills.json

data/history.json

data/ground\_truth.json

**Database:**

supabase\_client.py

**Frontend build tooling:**

frontend/package.json

frontend/vite.config.js

frontend/eslint.config.js

**Final Architecture Summary**

Frontend captures recruiter requirements and resume uploads. Backend extracts structured

resume data using hybrid parsing and NLP. Embedding and scoring engines compute semantic plus rule-based fit. XAI module generates explainable decision insights. Results are shown in candidate dashboards, exportable to CSV, and persisted in history. Optional communication modules generate and send candidate notifications.

### 5.3.6 Selected Source Code Snippets (Current Project)

This section includes selected real code snippets from important modules. It intentionally does not include every file from the project.

#### Snippet 1: API authentication decorator

**File:** `api/server.py`

```
def require_auth(fn):
    @wraps(fn)
    def _wrapper(*args, **kwargs):

        user_id, err = _resolve_authenticated_user_id()
        if err:
            status = 503 if "Supabase" in err else 401
            return jsonify({"error": err}), status
        g.user_id = user_id

        return fn(*args, **kwargs)
    return _wrapper
```

**Why this matters:** This centralizes user authentication across all protected API routes and returns consistent error codes.

#### Snippet 2: Multi-format resume text extraction

**File:** `core/text_extractor.py`

```
def extract_text(file: Any) -> str:

    filename = file.name.lower() if hasattr(file, "name") else ""
    if filename.endswith(".docx"):
        return _post_clean(_extract_from_docx(file))
    if filename.endswith(".txt"):

        return _post_clean(_extract_from_txt(file))
    if filename.endswith((".png", ".jpg", ".jpeg")):
        return _post_clean(_extract_with_ocr(file))
```

**Why this matters:** This handles multiple resume file types through a unified entry point with format-specific extractors.

### Snippet 3: Semantic scoring calibration

**File:** core/scoring.py

```
def compute_semantic_score(resume_embedding: Any, job_embedding: Any) -> float:
    if resume_embedding is None or job_embedding is None:
        return 0.0

    similarity = float(np.dot(resume_vec, job_vec))
    scaled_similarity = (similarity ** 2.5) if similarity > 0 else 0.0
    return _clamp(scaled_similarity, 0.0, 1.0)
```

**Why this matters:** This applies power-law calibration to cosine similarity for better score differentiation at the high end.

### Snippet 4: Frontend API client with token interceptor

**File:** frontend/src/api/client.js

```
const api = axios.create({ baseURL: '/api', headers: { 'Content-Type': 'application/json' } });

api.interceptors.request.use((config) => {

    const raw = localStorage.getItem('session');

    if (raw) {

        const token = JSON.parse(raw)?.access_token;

        if (token) config.headers.Authorization = `Bearer ${token}`;

        return config;

    }

});
```

**Why this matters:** This automatically attaches authentication tokens to all outgoing API requests without repetition in each call.

### Snippet 6: Authentication context state management

**File:** frontend/src/context/AuthContext.jsx

```
export function AuthProvider({ children }) {

    const [user, setUser] = useState(null);

    const [session, setSession] = useState(null);

    const [loading, setLoading] = useState(true);

    useEffect(() => {
```

```

const savedUser = localStorage.getItem('user');

const savedSession = localStorage.getItem('session');

if (savedUser && savedSession) {

  setUser(JSON.parse(savedUser));

  setSession(JSON.parse(savedSession));  }

setLoading(false);

}, []);  }

```

**Why this matters:** This keeps login state persistent across refresh and shares auth state globally.

### Snippet 7: Protected route guard logic

**File:** frontend/src/components/ProtectedRoute.jsx

```

export default function ProtectedRoute({ children }) {

  const { user, loading } = useAuth();

  if (loading) return <div className="page"><div className="spinner"></div></div>;

  if (!user) return <Navigate to="/login" replace />;

  return children;

}

```

**Why this matters:** This ensures only authenticated users can access private pages.

### Snippet 8: History persistence with retry support

**File:** utils/history\_store.py

```

def _execute_with_retry(operation, attempts=3, delay_seconds=0.6):

    last_error = None

    for attempt in range(attempts):

        try:

```



```

        return operation()

    except Exception as exc:

        last_error = exc

        if attempt < attempts - 1:

            time.sleep(delay_seconds)

        raise last_error

def save_history(job_data, threshold, shortlisted_candidates, all_results=None, user_id=None):

    if user_id is None or user_id == "guest" or supabase is None:

        return {"saved": False, "reason": "invalid_user_or_supabase_unavailable"}

```

**Why this matters:** This improves reliability when writing screening history to the database.

### Snippet 9: LLM email draft generation

**File:** core/communication\_engine.py

```

def generate_email_draft(candidate_data, job_data, email_type="rejection"):

    client = Groq(api_key=os.environ.get("GROQ_API_KEY"))

    candidate_name = candidate_data.get("resume_name", "Candidate")

    job_title = job_data.get("job_title", "Position")

    response = client.chat.completions.create(

        messages=[

            {"role": "system", "content": "You are a professional recruitment communication assistant."},

            {"role": "user", "content": prompt}  ], model=LLM_MODEL,)

    return response.choices[0].message.content.strip()

```

**Why this matters:** This automates recruiter communication with role-specific personalized drafts.

### Snippet 10: Structured explainability output helpers

**File:** core/xai\_engine\_v3.py

```

def _verdict(final_score: float) -> dict:

    if final_score >= 0.85:

```

```

if final_score >= 0.45:

    return {"label": "Moderate Fit", "color": "amber", "icon": "🟡"}

return {"label": "Weak Fit", "color": "red", "icon": "🔴"}

```

**Why this matters:** This provides clear decision bands for frontend explainability rendering.

## Snippet 11: Embedding engine with cloud and local fallback

**File:** `core/embedding_engine.py`

```

class EmbeddingEngine:

    def __init__(self, model_name: str = DEFAULT_MODEL):

        self.model = load_model(model_name)

        self.client = None

        if api_key:

            try:

                self.client = Groq(api_key=api_key)

            except Exception:

                self.client = None

    def embed_resume(self, text: str) -> np.ndarray:

        if self.client:

            response = self.client.embeddings.create(

                model="nomic-embed-text-v1.5",

                input=text, encoding_format="float"

            )

```

**Why this matters:** This supports high-quality cloud embeddings with robust fallback to local sentence-transformers.

## Snippet 12: Qualification level inference logic

**File:** core/qualification\_extractor.py

```
def _degree_level_from_text(degree_text: str) -> Optional[str]:

    t = _simplify_degree_token(degree_text)

    if any(k in t for k in ["DOCTORATE", "DOCTOROFPHILOSOPHY"]):

        return "doctorate"

    if any(_has_short(k) for k in ["MBA", "MCA", "LLM", "MSC", "MENG", "MCOM", "PG"]):

        return "masters"

    if any(k in t for k in ["BTECH", "BACHELOR", "BARCH", "BPHARM", "UNDERGRADUATE"]):

        return "bachelors"
```

**Why this matters:** This normalizes diverse degree patterns into consistent qualification levels.

## Snippet 13: CSV export for shortlisted candidates

**File:** utils/export\_utils.py

```
def export_to_csv(shortlisted_candidates: List[Dict], job_data: Dict) -> str:

    data = []

    for idx, candidate in enumerate(shortlisted_candidates, 1):

        scores = candidate.get("scores", {})

        row = { "Rank": idx,

            "Candidate_Name": candidate.get("resume_name", f"Candidate_{idx}"),

            "Phone_Contact": f"{' ' + candidate.get("phone")}{' ' if candidate.get("phone") else ''}",

            "Final_Score": scores.get("final_score", 0),

            "Matched_Skills": " ".join(scores.get("matched_skills", [])), }

        data.append(row)

    return pd.DataFrame(data).to_csv(index=False)
```

**Why this matters:** This creates recruiter-friendly exports while preserving phone formatting for spreadsheet tools.

## Snippet 14: Resume processing and ZIP extraction flow

**File:** frontend/src/pages/Processing.jsx

```
const handleFiles = async (newFiles) => {  
  
  for (const file of Array.from(newFiles)) {  
  
    if (file.type === 'application/zip' || file.name.endsWith('.zip')) {  
  
      const zip = new JSZip();  
  
      const extractedZip = await zip.loadAsync(file);  
  
      for (const [filename, fileData] of Object.entries(extractedZip.files)) {  
  
        if (!fileData.dir) {  
  
          const content = await fileData.async('arraybuffer');  
  
          const blob = new Blob([content], { type: getMimeType(ext) });  
  
          filesToAdd.push(blob);} } } ]
```

**Why this matters:** This enables bulk resume upload by accepting ZIP files and extracting valid documents client-side.

## Snippet 15: Navbar auth-aware navigation

**File:** frontend/src/components/Navbar.jsx

```
export default function Navbar() {  
  
  const { user, logout } = useAuth();  
  
  const [mobileMenuOpen, setMobileMenuOpen] = useState(false);  
  
  const handleLogout = () => {  
  
    logout();  
  
    navigate('/login');  
  
    setMobileMenuOpen(false);  
  
  };
```

**Why this matters:** This centralizes authenticated navigation behavior and responsive menu control.

## Snippet 16: Multi-layer skill extraction pipeline

**File:** `core/skill_extractor.py`

```
def extract_skills(text: str, skill_list: Optional[List[str]] = None,
                  threshold: int = 80) -> List[str]:

    if not text:

        return []

    if skill_list is None:

        skill_list = get_default_skills()

    aliases = get_skill_synonyms()
```

**Why this matters:** This combines direct matching, synonym expansion, fuzzy matching, and NLP validation for better recall.

## Snippet 17: Experience extraction with structural section mapping

**File:** `core/experience_extractor.py`

```
def map_resume_structure(text: str) -> List[Dict[str, Any]]:

    headers = []

    for cat, patterns in SECTION_MAP.items():

        for p in patterns:

            for match in re.finditer(p, text.lower()):

                headers.append({"type": cat, "start": match.start(), "content_start": match.end()})

def extract_date_intervals(text: str) -> List[Dict[str, Any]]:

    date_patterns = [

        rf"({month_names})\s*(\d{{2,4}})\s*[-\u2013\u2014to]+\s*({month_names}|{present_pattern})\s*(\d{{2,4}})?",

        rf"(\d{{1,2}})[/](\d{{2,4}})\s*[-\u2013\u2014to]+\s*(\d{{1,2}}|" + present_pattern + r")[/-]?" + rf"(\d{{2,4}})?",]
```

**Why this matters:** This reduces false positives by separating experience sections from project-only sections.

## Snippet 18: Candidate notification sender with SMTP fallback logic

**File:** `core/notification_engine.py`

```
def send_candidate_notifications(candidates, job_data, threshold):

    cfg = _smtp_config()

    if not (cfg["host"] and cfg["from_email"]):

        return {"sent": 0, "failed": 0, "skipped": len(candidates), "details": [], "error": "SMTP is not configured."}

    for candidate in candidates:

        score = float(candidate.get("scores", {}).get("final_score", 0) or 0)

        email_type = "next_steps" if score >= float(threshold) else "rejection"
```

**Why this matters:** This operationalizes automated shortlist/reject communication with robust fallback content.

## Snippet 19: Supabase client initialization

**File:** `supabase_client.py`

```
load_dotenv()

supabase = None

try

    url = os.environ.get("SUPABASE_URL", "")

    key = os.environ.get("SUPABASE_ANON_KEY", "")

    if url and key:

        supabase = create_client(url, key)

except Exception:

    supabase = None
```

**Why this matters:** This keeps backend services resilient when environment credentials are missing or invalid.

## Snippet 20: Candidates page shortlist and threshold logic

**File:** frontend/src/pages/Candidates.jsx

```
const shortlisted = useMemo(() =>
  results.filter(r => r.scores?.final_score >= threshold)
    .sort((a, b) => b.scores.final_score - a.scores.final_score),
  [results, threshold] );

const rejected = useMemo(() =>
  results.filter(r => r.scores?.final_score < threshold),
  [results, threshold] );
```

**Why this matters:** This gives recruiters dynamic decision control by recalculating shortlisted and rejected groups on threshold change.

## Snippet 21: NLP entity extraction and skill validation

**File:** core/nlp\_engine.py

```
class NLPEngine:

    def extract_entities(self, text: str) -> Set[str]:

        doc = self.nlp(text)

        candidates = set()

        if ent.label_ in ["ORG", "PRODUCT", "WORK_OF_ART", "GPE", "LANGUAGE"]:

            clean_ent = self._clean_term(ent.text)

            if self._is_valid_candidate(clean_ent):

                candidates.add(clean_ent)

        return candidates
```

**Why this matters:** This layer improves skill detection quality by combining spaCy NER with strict fuzzy validation.

## Snippet 22: Contact and profile extraction bundle

**File:** core/contact\_extractor.py

```
def extract_contact_info(text: str) -> Dict[str, str]:
```

```

if not text:

    return {"name": "", "email": "", "phone": "", "linkedin": "", "github": "", "portfolio": ""}

return {

    "name": extract_name(text),

    "email": extract_email(text),

    "phone": normalize_phone(extract_phone(text)),

    "linkedin": extract_linkedin(text),

    "github": extract_github(text),

    "portfolio": extract_portfolio(text)

}

```

**Why this matters:** This centralizes candidate identity and profile link extraction used in results and notifications.

### Snippet 23: LLM fallback extraction and merge strategy

**File:** core/hybrid\_extractor.py

```

def llm_extraction_fallback(text: str) -> dict:

    completion = client.chat.completions.create(

        model=LLM_MODEL,

        messages=[

            {"role": "system", "content": "You are a professional resume parser. Return raw JSON only."},

            {"role": "user", "content": prompt}

        ],

        temperature=0.0,

        response_format={"type": "json_object"}

    )

    return json.loads(completion.choices[0].message.content)

```

**Why this matters:** This improves robustness for difficult resumes when rule-based extraction misses key fields.



## Snippet 24: Job configuration submission logic

**File:** frontend/src/pages/JobConfig.jsx

```
const handleSubmit = async () => {

  if (!jobTitle || !jobDesc) { setError('Job title and description are required.');
```

return; }

```
  await saveJobConfig({ job_title: jobTitle,
    ...
    job_description: jobDesc,

    qualification: qualifications,

    year_of_passing: yearList,

    required_experience: experience,

    must_have_skills: mustHaveList,

    good_to_have_skills: goodList

  });

  navigate('/processing');
```

};

**Why this matters:** This converts recruiter inputs into validated payloads and starts the screening workflow.

## Snippet 25: History restore and re-screen actions

**File:** frontend/src/pages/History.jsx

```
const handleViewResults = async (record) => {

  const res = await getHistoryResults(record.id);

  if (res.data.results) {

    await setSessionData(res.data.job_data, res.data.results);

    navigate('/candidates');
```

}

```
  };

const handleRescreen = (record) => {
```

```

const jobData = {

  job_title: record.job_title || "",

  must_have_skills: record.must_have_skills || [],

  good_to_have_skills: record.good_to_have_skills || [],

  job_description: record.job_description || "",

};

setSessionData(jobData, []).then(() => navigate('/job-config'));

};

```

**Why this matters:** This supports workflow continuity by reopening old results and reusing old job setups.

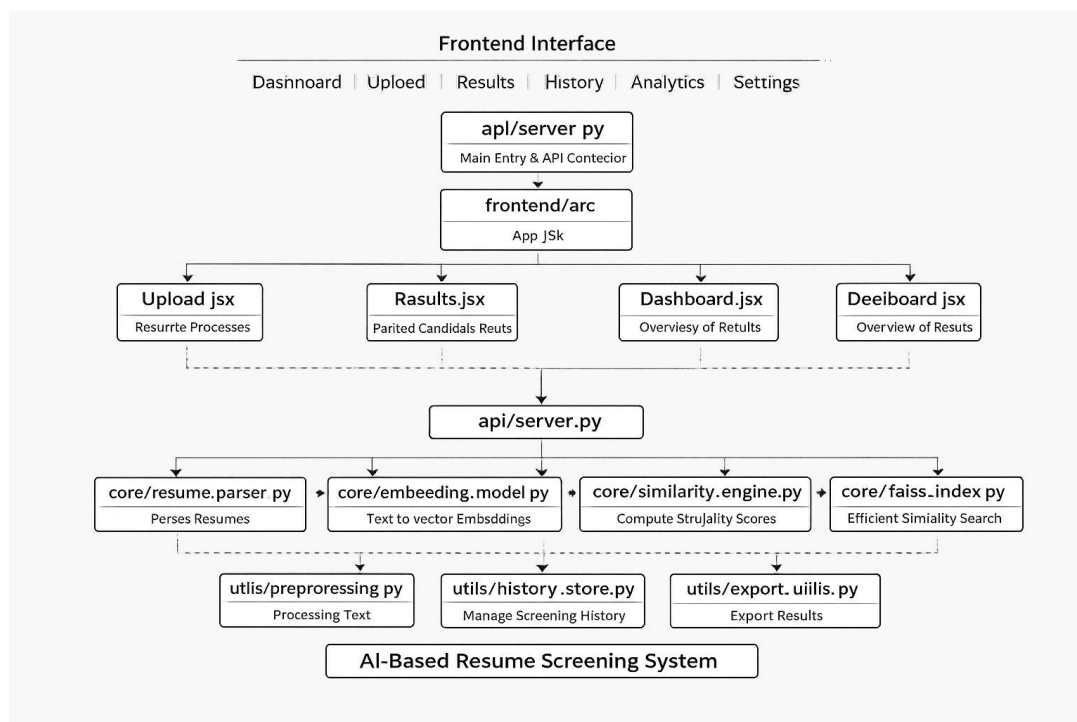


Fig 5.3.1 Layered Execution Flow of the AI-Based Smart Resume Screening System

The figure represents the layered architectural flow of the AI-Based Resume Screening System. It explains how the frontend interface guides the recruiter through different modules such as job configuration, resume upload, and result visualization. The system processes input resumes through parsing and preprocessing layers, followed by embedding generation using the LLaMA model. Further, similarity computation and ranking modules evaluate candidate

relevance, and the Explainable AI layer provides transparent reasoning for each decision. Finally, the results are displayed on the dashboard along with notifications sent to candidates.

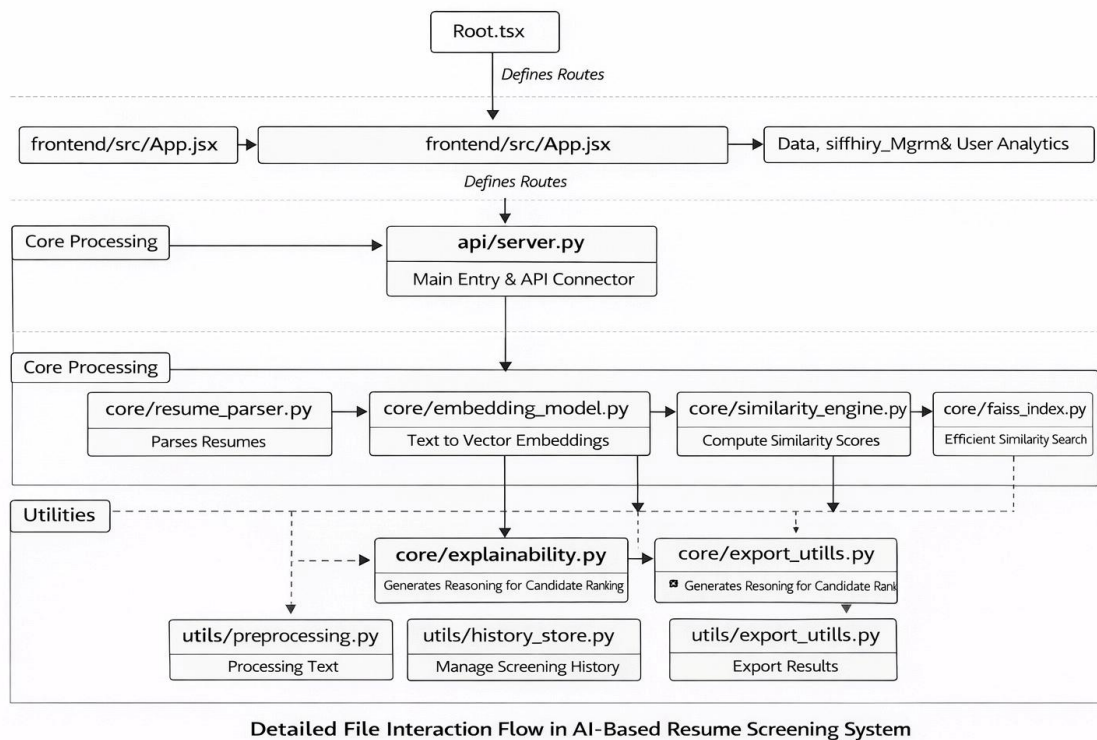


Fig 5.3.2 File Interaction Flow of the AI-Based Smart Resume Screening System

The figure illustrates the complete file-level interaction of the proposed AI-Based Resume Screening System. It highlights how frontend components such as App.jsx, Upload.jsx, and Results.jsx interact with the backend API server. The backend coordinates core modules including resume parsing, embedding generation, similarity computation, and ranking. Utility modules handle preprocessing, history management, and result export. Additionally, the explainability module generates reasoning for candidate ranking, and the notification service ensures communication with candidates. This interaction flow demonstrates how different system components collaborate to deliver efficient and intelligent resume screening.

## CHAPTER 6

### TESTING

#### 6.1 TEST CASES

Test cases were designed and executed to validate the major functionalities of the AI-Based Smart Resume Screening System. The purpose of these test cases was to ensure that the system performs accurately for various recruiter inputs and operational scenarios.

Table 6.1: Test Cases of AI-Based Smart Resume Screening System

| Test Case ID | Test Scenario          | Input                         | Expected Output                           | Result |
|--------------|------------------------|-------------------------------|-------------------------------------------|--------|
| TC01         | Upload Resume          | Valid resume file (PDF/DOCX)  | Resume should be uploaded successfully    | Passed |
| TC02         | Upload Job Description | Valid job description details | Job criteria should be saved successfully | Passed |
| TC03         | Resume Parsing         | Uploaded resume document      | Candidate details extracted correctly     | Passed |
| TC04         | Candidate Screening    | Resumes + job description     | Ranked candidate list displayed           | Passed |
| TC05         | Skill Matching         | Resume with required skills   | Matching score generated accurately       | Passed |
| TC06         | Explainable AI Output  | Selected candidate result     | Score breakdown and reasoning displayed   | Passed |
| TC07         | Email Notification     | Shortlisted candidate email   | Notification sent successfully            | Passed |

#### 6.2 TESTING METHODS

##### 6.2.1 Unit Testing

Unit testing improved the reliability of individual modules and reduced the possibility of errors in later stages of testing. Unit Testing is the process of testing individual components or functions of the system independently to ensure that each unit performs correctly. It focuses on the smallest testable parts of the application, such as a function, method, or module.

In the AI-Based Smart Resume Screening System, unit testing was used to verify the correctness of core functionalities such as resume parsing, text preprocessing, embedding generation, similarity computation, candidate ranking, and the Explainable AI module. This

helped in detecting errors at an early stage before integrating all modules into the complete system.

Examples of units tested in the project include:

- validation of resume file upload (PDF format handling)
- extraction of text from resumes,
- text preprocessing and cleaning functions,
- generation of embeddings using transformer models,
- similarity score calculation between resume and job description,
- ranking logic for candidate selection,
- explainability module for generating reasoning (skill matching and justification),
- handling of empty or invalid inputs.

Unit testing improved the reliability of individual modules and reduced the possibility of errors in later stages of testing

### **6.2.2 Integration Testing**

Integration Testing is performed to verify whether different modules of the system work correctly when combined together. Even if each module functions properly on its own, issues may arise when modules interact. Integration testing helps identify and resolve such problems. In the AI-Based Smart Resume Screening System, integration testing was carried out to ensure smooth communication between the frontend interface, backend processing modules, and AI components. It verified that data flows correctly across different stages of the pipeline and that the system functions as a unified whole.

The following integrations were tested:

- connection between resume upload interface and backend processing modules,
- integration of resume parser with preprocessing and embedding generation,
- flow of embeddings into similarity computation and ranking engine,
- integration of similarity results with the explainability module,
- communication between backend APIs and frontend result display,
- export and storage of processed screening results.

This testing ensured that all modules were properly connected and that the application delivered accurate results with a smooth and consistent user experience.

### **6.2.3 System Testing**

System Testing is conducted to evaluate the complete and integrated software application as a whole.

System testing verified that:

- The application meets functional requirements
- The system behaves correctly under different user scenarios
- The complete workflow executes without failure
- The user interface responds properly across all modules
- 

## **6.3 RESULTS AND ANALYSIS**

The testing process showed that the AI-Based Smart Resume Screening System performed successfully across its major modules. Unit Testing confirmed that individual components such as resume parsing, embedding generation, similarity computation, and explainability operated correctly. White Box Testing verified that the internal logic and execution flow of the AI pipeline were accurate. Black Box Testing confirmed that the system produced expected outputs for various user inputs. Integration Testing ensured that all modules communicated effectively and functioned properly as a complete system.

The results of testing indicated that:

- resume upload and text extraction were performed successfully,
- embeddings were generated correctly for resumes and job descriptions,
- similarity scores accurately reflected candidate relevance,
- candidate ranking was consistent and reliable,
- explainable AI provided meaningful reasoning for results,
- frontend and backend integration functioned smoothly.

Overall, the system met the expected functional requirements and demonstrated accurate, reliable, and efficient performance during testing.

# CHAPTER 7

## RESULTS&DISSCUSSIONS

The proposed Explainable AI-Based Smart Resume Screening System was implemented as a web-based intelligent application designed to automate and enhance the recruitment process. The system integrates multiple AI-driven modules such as resume parsing, text preprocessing, semantic embedding generation, similarity computation, and candidate ranking. In addition, an Explainable AI (XAI) component is incorporated to provide transparent reasoning behind each screening decision.

The system enables recruiters to define job requirements, upload resumes in bulk, process candidate data automatically, and obtain ranked results along with detailed explanations. This reduces manual effort, improves screening accuracy, and ensures fairness and consistency in hiring decisions.

### 7.2 OUTPUT SCREENSHOTS

#### 7.2.1 Home Interface (Landing Page)

The home interface serves as the entry point of the application and provides an overview of the system's capabilities. It highlights key features such as semantic matching, automated skill extraction, and multi-factor candidate scoring.

The interface also visually explains the workflow of the system, which includes defining job requirements, uploading resumes, AI-based processing, and reviewing results. A "Get Started" option allows users to quickly initiate the screening process. This page is designed to provide clarity, usability, and easy navigation for recruiters.

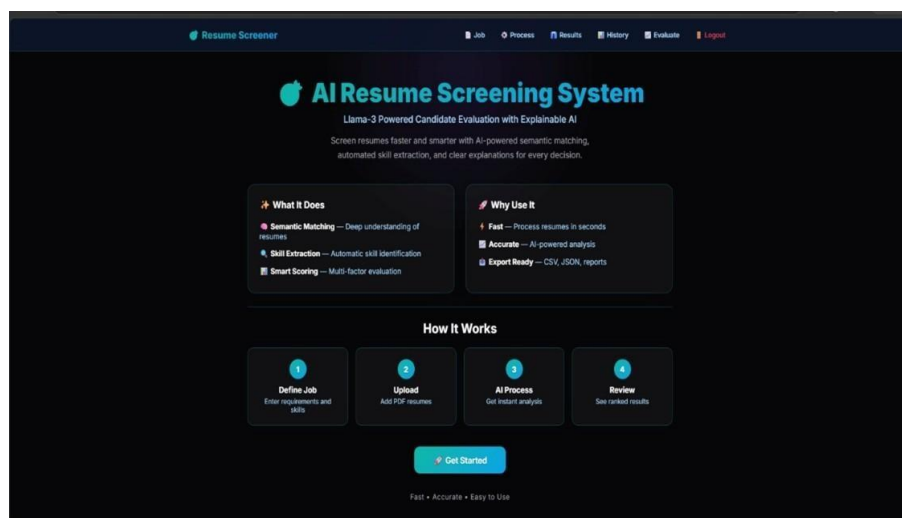


Figure 7.1.1: Home Interface of Resume Screening System

The system also supports additional inputs such as preferred qualifications and complementary skills, allowing more flexible and precise job definitions. These inputs are stored and later used during the AI-based matching process to compare candidate resumes with job expectations. This module plays a crucial role in ensuring accurate screening, as the quality of input directly affects the relevance of candidate ranking

The figure displays two screenshots of the 'Job Configuration' interface. The left screenshot shows the form with the following sections: 'Job Title' (a text input field), 'Job Description' (a large text area), 'Job Requirements' (a section with a table for 'Required Qualifications' and a 'Required Experience' field), and 'Skills Requirements' (a section with a table for 'Required Skills'). The right screenshot shows the same form, but with a warning message in the 'Job Description' field: 'We noticed you've entered a very long Job Description. This could impact the system's ability to process the information. Please keep it under 1000 characters.' The 'Job Requirements' and 'Skills Requirements' sections are also visible on the right screenshot.

Figure 7.1.2: Job Configuration Interface & Input

## 7.2.2 Resume Upload and Processing Interface

The resume processing module enables users to upload multiple resumes simultaneously in various formats such as PDF, DOCX, and ZIP files. The system supports drag-and-drop functionality for ease of use.

Once uploaded, the system automatically extracts text from resumes, preprocesses the content by removing noise, and standardizes the data for further analysis. This includes tokenization, normalization, and structuring of candidate information such as skills, education, and experience.

The interface also displays upload status, number of files processed, and completion confirmation, ensuring transparency during the processing stage.



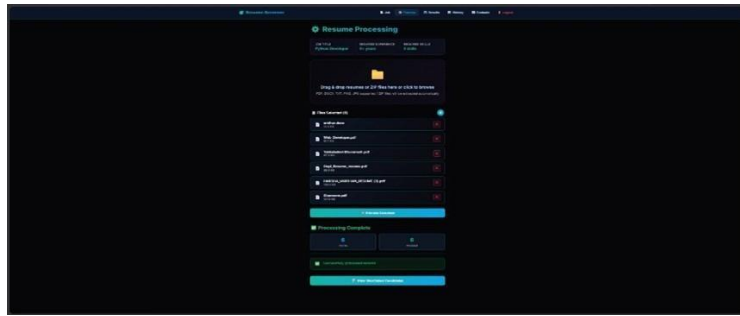


Figure 7.1.3: AI-Based Resume Input

### 7.2.3 AI-Based Resume Screening

After preprocessing, the system applies AI techniques to evaluate candidate resumes. The processed text is converted into vector embeddings using embedding models, enabling semantic understanding of both resumes and job descriptions.

The similarity engine computes relevance scores by comparing candidate embeddings with job requirement embeddings. Based on these scores, candidates are ranked and filtered according to predefined thresholds.

This module ensures accurate matching beyond simple keyword comparison by understanding context and meaning, thereby improving the quality of candidate selection.

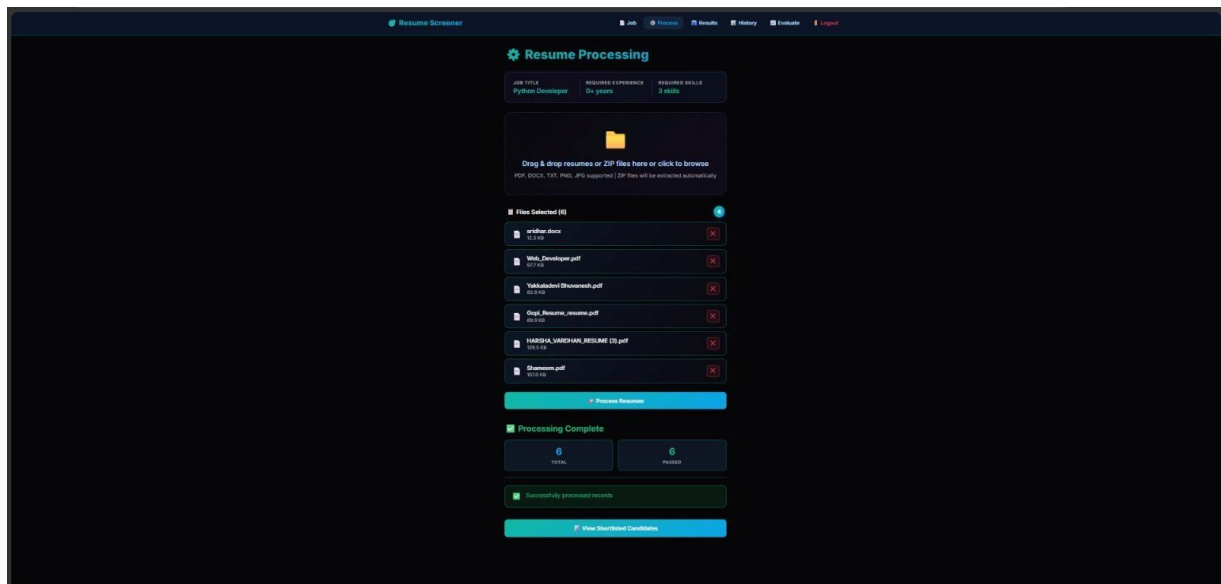


Figure 7.1.4: AI-Based Resume Processing

### 7.2.4 Candidate Ranking and Results Dashboard

The results dashboard presents the shortlisted candidates along with their ranking, similarity scores, and evaluation metrics. Users can adjust screening thresholds to control the number of shortlisted candidates and compare multiple candidates side by side.

The interface also provides summary statistics such as total candidates processed, number of shortlisted candidates, and average match score. This helps recruiters quickly analyze the screening outcome and make informed decisions.

Export options such as CSV download and notification features are also included for further processing and communication.

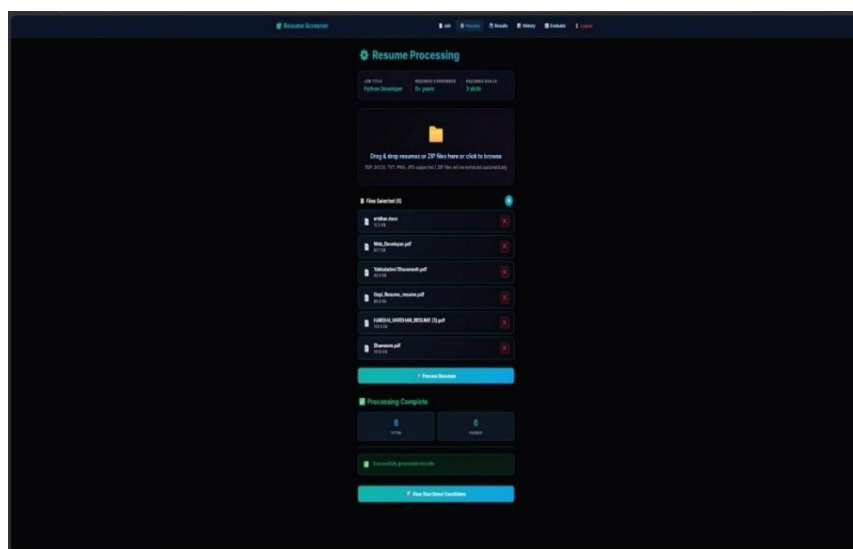


Figure 7.1.5: Shortlisted Candidates Dashboard

### 7.2.5 Explainable AI-Based Candidate Evaluation

A key feature of the system is the integration of Explainable AI, which provides detailed reasoning behind each candidate's ranking. The system generates insights such as skill match percentage, experience relevance, confidence level, and justification for selection or rejection. It also highlights strengths, gaps, and recommended next steps such as interview suggestions. This transparency helps recruiters understand the decision-making process of the AI model and increases trust in automated screening. Explainability ensures that the system is not only accurate but also interpretable and accountable

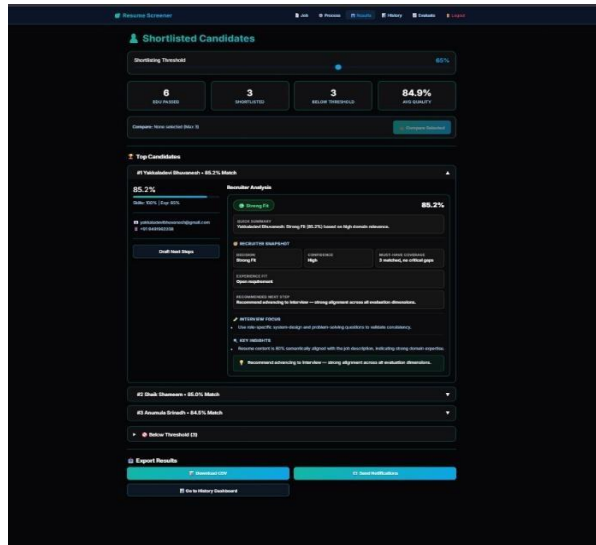


Figure 7.1.6: Explainable AI Candidate Analysis

## 7.1.6 Screening History Module

The history module maintains a record of previous screening sessions, allowing users to revisit past evaluations. Each entry includes job details, timestamp, and screening results. This feature supports comparison of different hiring sessions, tracking of candidate evaluations over time, and reuse of previous configurations. It enhances usability by providing continuity and auditability in the recruitment process.

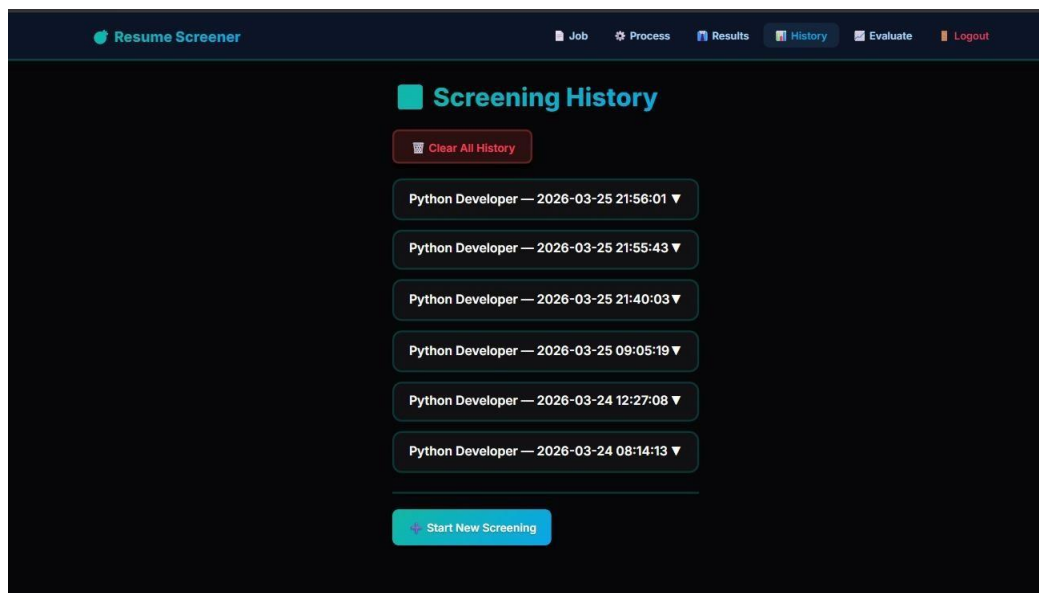


Figure 7.1.7: Resume Screening History

### 7.3 PERFORMANCE ANALYSIS

The proposed AI-Based Smart Resume Screening System demonstrated effective performance during implementation and testing. The intelligent screening engine successfully evaluated resumes based on semantic similarity, skill matching, and experience relevance.

The system efficiently ranked candidates and maintained acceptable response time during resume processing and result generation.

Performance observations include:

- Fast resume screening and candidate ranking response time
- Accurate candidate matching based on job requirements
- Effective skill and experience evaluation using multi-criteria scoring
- Smooth frontend-backend interaction with AI model integration
- Reliable dashboard performance and result visualization

The results indicate that the system efficiently performs intelligent resume screening while maintaining usability, scalability, and responsiveness.

### 7.4 COMPARISON WITH EXISTING SYSTEM

To evaluate the effectiveness of the proposed system, a qualitative comparison was made between traditional resume screening methods and the AI-based screening approach.

**Table 7.1:** Comparison of Traditional and Proposed Resume Screening Systems

| Metric          | Traditional Screening              | AI-Based Resume Screening                   |
|-----------------|------------------------------------|---------------------------------------------|
| Screening Speed | Manual and time-consuming          | Fast automated processing                   |
| Accuracy        | Limited explanation                | Explainable AI-based insights               |
| Transparency    | Manual planning requires more time | Automated AI scheduling improves efficiency |
| Scalability     | Limited handling capacity          | Handles large volumes efficiently           |

The comparison shows that the proposed system significantly improves study consistency, workload distribution, efficiency, and personalization compared to traditional static planning methods.

## **CHAPTER 8**

### **CONCLUSION AND FUTURE WORK**

#### **8.1 SUMMARY OF THE PROJECT**

The proposed AI-Based Smart Resume Screening System was developed to provide organizations with an intelligent and efficient solution for candidate evaluation and recruitment management. Unlike traditional resume screening methods, the system automatically analyzes resumes using semantic understanding, skill matching, and contextual experience evaluation to identify the most suitable candidates.

The system integrates advanced AI techniques such as LLaMA-based embeddings, multi-criteria candidate scoring, explainable AI functionalities, and automated communication modules to enhance recruitment efficiency and transparency. By combining semantic resume analysis, weighted skill assessment, ranking intelligence, and recruiter-friendly dashboards, the system transforms conventional hiring processes into an intelligent talent acquisition framework.

Overall, the developed solution helps recruiters reduce manual effort, accelerate hiring decisions, improve candidate shortlisting accuracy, ensure fair evaluation, and manage recruitment workflows with greater efficiency and confidence.

#### **8.2 ACHIEVEMENTS**

The major achievements of the proposed AI-Based Smart Resume Screening System include:

- Successful development of an intelligent AI-driven resume screening system
- Implementation of semantic resume analysis using LLaMA embeddings
- Development of multi-criteria candidate ranking and scoring mechanism
- Integration of weighted skill matching and experience evaluation
- Implementation of Explainable AI for transparent candidate recommendations
- Development of automated email notification for candidate updates
- Successful integration of frontend, backend, and AI processing modules
- Creation of an interactive and user-friendly web-based recruiter dashboard

## 8.3 LIMITATIONS

- Performance depends on the quality and format of uploaded resumes
- Difficulty in extracting data from poorly structured or image-based resumes
- Semantic matching accuracy depends on model training and domain relevance
- Limited understanding of highly specialized or niche industry skills
- Bias may occur if training data contains historical recruitment patterns
- Experience evaluation may not fully capture real-world candidate capability
- Requires significant computational resources for embedding generation and ranking
- Internet and server dependency for smooth web-based operation
- Automated email notifications depend on valid candidate contact details
- Frequent updates needed to match changing job market skills and trends
- Final hiring decisions still require human recruiter validation
- Integration with external ATS or job portals may need additional customization

## 8.4 FUTURE ENHANCEMENTS

Several enhancements can be incorporated in future versions of the proposed system to improve functionality and scalability

- Secure multi-user authentication and role-based access
- Native mobile application with push notifications
- Cloud synchronization for cross-device access
- Integration with job portals and ATS platforms
- Real-time hiring analytics dashboard
- Explainable AI with bias and fairness detection
- Skill-gap identification and analysis
- Adaptive screening based on job requirements
- Automated interview question generation
- Predictive candidate success analysis

## REFERENCES

- [1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed., Upper Saddle River, NJ, USA: Pearson Education, 2010.
- [2] J. Devlin, M. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proceedings of NAACL-HLT*, 2019.
- [3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proceedings of EMNLP*, 2019.
- [4] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Transactions on Big Data*, 2019.
- [5] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, “Distributed Representations of Words and Phrases and Their Compositionality,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.
- [6] M. T. Ribeiro, S. Singh, and C. Guestrin, “Why Should I Trust You? Explaining the Predictions of Any Classifier,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [7] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*, O’Reilly Media, 2009.
- [8] A. Vaswani et al., “Attention Is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [9] G. Salton and C. Buckley, “Term-Weighting Approaches in Automatic Text Retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [10] T. Kenter and M. de Rijke, “Short Text Similarity with Word Embeddings,” in *Proceedings of CIKM*, 2015.
- [11] S. Palshikar, “Keyword Extraction from Text Using Graph-Based Methods,” *Proceedings of I-KNOW*, 2007.
- [12] Y. Liu et al., “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” 2019.